« Établissement à compléter »

****

« Faculté / École à compléter »

****

« Département à compléter »

OPTIMISATION DE L'ARCHITECTURE SDN DÉCENTRALISÉE POUR UNE INFRASTRUCTURE DISTRIBUÉE CLOUD EDGE

MÉMOIRE DE MASTER

Présenté et soutenu par :

Léocadie

En vue de l'obtention du :

« Intitulé du diplôme à compléter »

Sous la direction de :

« Nom du directeur de mémoire à compléter »

Devant le jury composé de :

« Composition du jury à compléter »

Année académique : 2025 - 2026

RÉSUMÉ

Les infrastructures Cloud-Edge décentralisées confrontent le pilotage des réseaux définis par logiciel (SDN) à une difficulté nouvelle : les contrôleurs n'y sont plus homogènes. Un nœud Edge aux ressources contraintes côtoie un centre de données Cloud puissant mais éloigné, et la latence entre commutateurs et contrôleurs devient un facteur déterminant. L'architecture SDN_CTRLML proposée par Aly et al. (2025) répond au problème d'équilibrage de charge entre contrôleurs par une couche de planification sans liaison fixe, mais elle repose sur une métrique unique, l'utilisation CPU, et suppose des contrôleurs identiques. Ces deux hypothèses ne tiennent plus en environnement Cloud-Edge.

Ce mémoire propose SDN_CTRLML-Edge, une extension de cette architecture. La contribution combine trois éléments : une observation multi-métriques (utilisation CPU, mémoire, bande passante, latence, nombre de flux et intensité Packet-In), un lissage par moyenne mobile exponentielle qui élimine le bruit des mesures avant décision, et une classification dynamique par K-means++ répartissant les contrôleurs en trois profils de charge. Une couche décisionnelle à deux régimes exploite cette classification : en fonctionnement normal, le flux est confié au contrôleur sain le plus proche en latence ; en saturation généralisée, le mécanisme bascule vers un délestage pur vers le contrôleur le moins chargé.

La solution a été réalisée sous la forme d'un simulateur en Python et évaluée sur 1 000 exécutions indépendantes croisant cinq stratégies, huit niveaux de charge et vingt-cinq graines aléatoires, avec intervalles de confiance à 95 % et tests de Wilcoxon appariés. Par rapport à une liaison statique de type ONOS, l'approche proposée réduit le temps de réponse moyen de 57,3 % et augmente le débit de 83,3 %. Face à SDN_CTRLML, elle réduit le temps de réponse de 11,3 % et le nombre de réaffectations de 35,9 %, tout en portant l'indice d'équité de Jain à un plateau de 0,993. L'ensemble de ces écarts est statistiquement significatif au seuil de 5 %.

Mots-clés : SDN décentralisé, Cloud-Edge, équilibrage de charge, K-means++, moyenne mobile exponentielle, apprentissage automatique.

ABSTRACT

Decentralised Cloud-Edge infrastructures confront Software-Defined Networking (SDN) with a new difficulty: controllers are no longer homogeneous. Resource-constrained Edge nodes coexist with powerful but distant Cloud data centres, and the latency between switches and controllers becomes a decisive factor. The SDN_CTRLML architecture proposed by Aly et al. (2025) addresses controller load balancing through a bindingless scheduling layer, but it relies on a single metric, CPU utilisation, and assumes identical controllers. Neither assumption holds in a Cloud-Edge environment.

This thesis proposes SDN_CTRLML-Edge, an extension of that architecture. The contribution combines three elements: multi-metric monitoring (CPU, memory, bandwidth, latency, number of flows and Packet-In intensity), exponential moving average smoothing that removes measurement noise before any decision, and dynamic K-means++ clustering that groups controllers into three load profiles. A two-regime decision layer exploits this classification: under normal operation, a flow is assigned to the nearest healthy controller in terms of latency; under generalised saturation, the mechanism switches to pure load shedding towards the least loaded controller.

The solution was implemented as a Python simulator and evaluated over 1,000 independent runs crossing five strategies, eight load levels and twenty-five random seeds, with 95 % confidence intervals and paired Wilcoxon tests. Compared with a static ONOS-style binding, the proposed approach reduces mean response time by 57.3 % and increases throughput by 83.3 %. Against SDN_CTRLML, it reduces response time by 11.3 % and controller reassignments by 35.9 %, while raising Jain's fairness index to a plateau of 0.993. All these differences are statistically significant at the 5 % level.

Keywords: decentralised SDN, Cloud-Edge, load balancing, K-means++, exponential moving average, machine learning.

Liste des abréviations

Abréviation	Signification
API	Application Programming Interface
BLAC	Balanced Load Adaptive Controller
CPU	Central Processing Unit
DDoS	Distributed Denial of Service
DOE	Design of Experiments (plan d'expériences)
EMA	Exponential Moving Average (moyenne mobile exponentielle)
IaaS	Infrastructure as a Service
IC95	Intervalle de confiance à 95 %
IoT	Internet of Things (Internet des objets)
K-means++	Algorithme de partitionnement à initialisation optimisée
LSTM	Long Short-Term Memory
MEC	Mobile Edge Computing
ML	Machine Learning (apprentissage automatique)
M/M/1	File d'attente markovienne à un serveur
NBI	Northbound Interface
NFV	Network Function Virtualization
ONF	Open Networking Foundation
ONOS	Open Network Operating System
PaaS	Platform as a Service
QoS	Quality of Service (qualité de service)
RL	Reinforcement Learning (apprentissage par renforcement)
SaaS	Software as a Service
SBI	Southbound Interface
SDN	Software-Defined Networking (réseau défini par logiciel)
SDN_CTRLML	Architecture de référence d'Aly et al. (2025)
SDN_CTRLML-Edge	Architecture proposée dans ce mémoire
TLS	Transport Layer Security
VM	Virtual Machine (machine virtuelle)

LISTE DES FIGURES

Figure 1 : Réseau traditionnel et réseau SDN.....3
Figure 2 : Les trois couches de l'architecture SDN.....6
Figure 3 : Plans et interfaces de l'architecture SDN.....7
Figure 4 : Fonctionnement du protocole OpenFlow.....9
Figure 5 : Architectures du SDN.....11
Figure 6 : Déchargement des traitements vers le serveur Edge.....18
Figure 7 : Architecture de l'Edge Computing.....20
Figure 8 : Pipeline de fonctionnement de la solution SDN_CTRLML-Edge.....45
Figure 9 : Schéma de la prise de décision (Étape 5).....48
Figure 10 : Logigramme de la règle de décision à deux régimes.....50
Figure 11 : Pipeline expérimental complet.....53
Figure 12 : Comparaison des moyennes globales de temps de réponse et de débit.....57
Figure 13 : Temps de réponse moyen en fonction du taux d'arrivée.....58
Figure 14 : Débit servi en fonction du taux d'arrivée.....59
Figure 15 : Indice d'équité de Jain en fonction du taux d'arrivée.....60
Figure 16 : Utilisation CPU moyenne des contrôleurs.....61
Figure 17 : Nombre de réaffectations en fonction du taux d'arrivée.....62
Figure 18 : Contribution de chaque composant au temps de réponse et à l'équilibrage.....–
Figure 19 : Évolution de l'équité au fil des cycles de décision, avec et sans rééquilibrage.....–
Figure 20 : Répartition de la charge de contrôle avant et après rééquilibrage (6 contrôleurs)…–

LISTE DES TABLEAUX

Tableau 1 : Tableau comparatif des approches ML dans le SDN.....39

Tableau 2 : Décisions de la couche décisionnelle selon la situation observée.....45

Tableau 3 : Interprétation des clusters de charge.....47

Tableau 4 : Environnement matériel et logiciel.....54

Tableau 5 : Bibliothèques Python utilisées.....55

Tableau 6 : Paramètres du plan d'expériences.....56

Tableau 7 : Moyennes globales par stratégie sur les 1 000 exécutions.....57

Tableau 8 : Tests de Wilcoxon appariés entre la stratégie proposée et les références.....63

Tableau 9 : Étude d'ablation : contribution mesurée de chaque composant.....–

Tableau 10 : Résultats de l'émulation sur commutateurs Open vSwitch réels.....–

SOMMAIRE

INTRODUCTION GÉNÉRALE.....1

CHAPITRE I : CONTEXTE ET PROBLÉMATIQUE.....2

CHAPITRE II : MÉTHODOLOGIE.....33

CHAPITRE III : RÉSULTATS ET COMMENTAIRES.....55

CONCLUSION GÉNÉRALE.....66

RÉFÉRENCES BIBLIOGRAPHIQUES.....68

ANNEXES.....69

TABLE DES MATIÈRES.....71

Optimisation de l'architecture SDN décentralisée pour une infrastructure distribuée cloud Edge

INTRODUCTION GENERALE

Dans un monde de plus en plus interconnecté, les avancées technologiques telles que l'Internet des Objets (IoT), la virtualisation des fonctions réseau (NFV) et le Mobile Edge Computing (MEC) redéfinissent les paradigmes de l'informatique distribuée. Ces évolutions imposent un rapprochement des services cloud vers les utilisateurs finaux, afin de minimiser les latences et d'optimiser la gestion des flux de données en temps réel. Cette dynamique donne naissance aux infrastructures cloud distribuées (Distributed Cloud Infrastructure – DCI), déployées sur de multiples sites ou Points of Presence (PoPs), qui visent à décentraliser les ressources computationnelles pour une plus grande résilience et scalabilité.

Parallèlement, le développement fulgurant de l'Internet des Objets Étendu (IoE) accentue ces enjeux. Le nombre d'appareils intelligents connectés à Internet explose, générant des volumes massifs de données à grande échelle. Des domaines variés, tels que les maisons intelligentes et les véhicules autonomes dans les transports, les caméras de surveillance ou encore les robots intelligents en fabrication industrielle, illustrent cette prolifération. Selon les estimations récentes de l'IoT Analytics en octobre 2025, le nombre d'appareils IoT connectés atteint 21,1 milliards en 2025, en croissance de 14 % par rapport à 2024. Cette estimation prévoit une accélération à 39 milliards d'ici 2030. Ce qui amplifie les défis inhérents aux modèles traditionnels de cloud computing centralisé. Ces derniers se heurtent à des problèmes critiques : surcharge de la bande passante, lenteur des temps de réponse, vulnérabilités sécuritaires accrues et insuffisante protection de la vie privée. Le cloud computing classique, bien qu'efficace pour le traitement centralisé à grande échelle, peine désormais à répondre aux exigences d'une société intelligente en perpétuelle évolution, où la rapidité, la fiabilité et la confidentialité des données sont primordiales.

C'est dans ce contexte que les technologies d'edge computing émergent comme un paradigme innovant. Contrairement au cloud centralisé, l'edge computing privilégie les calculs à la périphérie du réseau, au plus près des sources de données et des utilisateurs. Ce modèle,

léger en termes de stockage et de traitement local à petite échelle, permet une réduction significative des transferts de données vers des centres distants, tout en améliorant la réactivité et la sécurité. Les appareils périphériques, omniprésents dans notre quotidien, deviennent ainsi des nœuds actifs d'un écosystème distribué, favorisant une architecture hybride cloud-edge capable de supporter des applications critiques comme la conduite autonome ou la maintenance prédictive en temps réel.

Au cœur de cette transformation se positionne le Software-Defined Networking (SDN), dont les architectures décentralisées offrent une flexibilité accrue pour orchestrer les flux dans ces environnements hétérogènes. L'optimisation de ces architectures SDN décentralisées pour les infrastructures distribuées cloud-edge représente un enjeu stratégique, visant à concilier performance, scalabilité et robustesse face aux contraintes de latence et de ressources limitées. Ce mémoire explore précisément ces dimensions, en proposant une analyse approfondie des mécanismes d'optimisation et en évaluant leur impact sur l'efficacité globale des systèmes distribués. À travers cette étude, nous ambitionnons de contribuer à l'avènement d'une informatique edge-centric plus agile et sécurisée, adaptée aux défis de la prochaine génération de réseaux intelligents.

L'évolution rapide des réseaux informatiques, caractérisée par une augmentation exponentielle du volume de trafic, une plus grande hétérogénéité des équipements et des exigences toujours plus strictes en termes de performance, de QoS et de sécurité, a révélé les limites des architectures réseaux traditionnelles. Le Software-Defined Networking (SDN) a émergé comme une solution prometteuse en séparant le plan de contrôle du plan de données, offrant ainsi une programmation centralisée et une visibilité globale du réseau.

CHAPITRE I : CONTEXTE ET PROBLEMATIQUE

Dans ce chapitre, nous allons parcourir tour à tour de la généralité de l'architecture SDN et la généralité des infrastructures Cloud Edge.

I. GENERALITES SUR LES SDN

1. Définition du SDN

Le terme SDN signifie Software-Defined Networking qui se traduit par réseau défini par logiciel. Le SDN est une nouvelle approche d'architecture réseau où le plan de contrôle (décisions de routage) est séparé du plan de données (transmission des paquets) permettant une gestion centralisée et programmable du réseau via des contrôleurs logiciels. Le plan de contrôle est déployé sur des plateformes dont les capacités sont plus grandes que celles des commutateurs réseaux classiques.

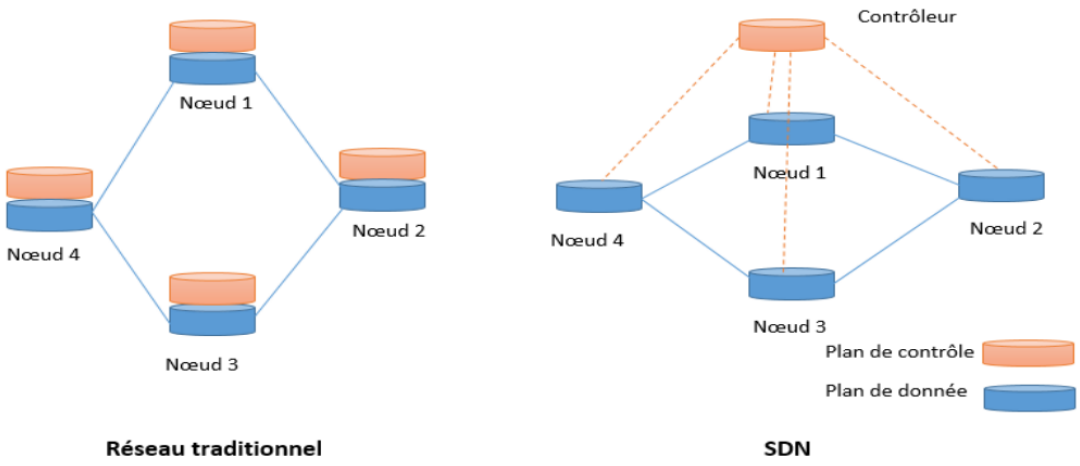


***Figure 1** : Réseau traditionnel et réseau SDN*

Selon L'ONF le SDN est une architecture qui sépare le plan de contrôle du plan de données, et unifie les plans de contrôle de plusieurs périphériques dans un seul software de contrôle externe appelé « Contrôleur », qui voit le réseau dans sa totalité pour gérer l'infrastructure via des interfaces de communications appelées APIs. Le contrôleur en question fait abstraction de la couche physique pour les applications qui communiquent en langage développeur, permettant la programmation du réseau [3].

Le SDN est plus globalement reconnu aujourd'hui comme une architecture permettant d'ouvrir le réseau aux applications car il crée un dialogue entre le réseau et les applications.. **Ce qui intègre les deux volets ci-dessous :**

- **Permettre aux applications de programmer le réseau afin d'en accélérer le déploiement : les applications peuvent directement demander au réseau ce dont elles ont besoin comme ressources ;**
- **Permettre au réseau de mieux identifier les applications transportées pour mieux les gérer (QoS, sécurité, …) : le réseau devient capable d'identifier quelle application génère quel trafic et peut adapter son comportement en conséquence.**

2. Architecture du SDN

L'architecture SDN utilise des interfaces de programmation d'application (Application Programming Interfaces APIs) standardisées pour permettre aux programmeurs réseau de définir et de reconfigurer rapidement la manière dont les données ou les ressources sont gérées. Les applications réseau peuvent reconfigurer le réseau et ses composants ou récupérer des données spécifiques selon leurs besoins à l'aide des API.

L'ONF (Open Networking Foundation) décrit une architecture de haut niveau du SDN, qui est fonctionnellement et verticalement divisée en trois couches :

- La couche infrastructure : appelée aussi **plan de données**, elle est composée des équipements d'acheminement ou de transfert tels que les switches ou les routeurs ; son rôle principal étant de transmettre les données et collecter les statistiques.

- La couche de contrôle : appelée aussi **plan de contrôle**, elle est constituée principalement d'un ou plusieurs contrôleurs SDN, son rôle est de contrôler et de gérer les équipements de l'infrastructure à travers une interface appelée '**south-bound API'**.

- La couche application : représente les applications qui permettent de déployer de nouvelles fonctionnalités réseau, comme l'ingénierie de trafic, QoS, la sécurité, etc. Ces applications sont construits moyennant une interface de programmation appelée '**north-bound API**'

L'architecture SDN est définie par trois abstractions [3]

- **L'abstraction de transmission** : permet aux applications de programmer et de prendre les décisions concernant le réseau sans pour autant connaitre les détails de

l'infrastructure physique. Ceci est achevé à travers l'usage de protocoles ouvert et standardisé pour la communication avec les équipements réseau.

- **L'abstraction de distribution** : elle est implémentée par le contrôleur et est essentiellement responsable de deux taches. La première est d'insérer les règles de transfert dans les switchs. La seconde est de récupérer les informations de la couche inférieure, pour informer les applications de l'état de celle-ci, pour pouvoir former une vue globale du réseau
- **L'abstraction de spécification** : permet aux applications d'exprimer le comportement désiré du réseau sans en être directement responsable

Les couches du SDN communiquent entre elles au travers des interfaces API suivantes:

➢ **Interfaces Sud**

Les interfaces Sud ou (Southbound APIs) représentent les interfaces de communication qui permettent au contrôleur SDN d'interagir avec les équipements de la couche application, tel que les switches, et les routeurs. Au travers de ces interfaces, le contrôleur injecte les différente politique aux équipements et récupère les informations permettant aux applications de construire une vue globale du réseau. Le protocole le plus utilisé, et le plus déployé comme interface Sud est le protocole OpenFlow, qui a été standardisé par l'ONF. Il existe d'autres alternatives d'interface Sud, tels que ForCes, ou Open vSwitch Database (OVSDB), mais le protocole openflow est actuellement le standard de facto, qui est largement accepté et répandu dans les réseaux SDN [2].

➢ **Interfaces Nord**

Il s'agit de l'interface permettant la communication entre le contrôleur et la couche application. Les interfaces Nord ou North-bound servent à programmer les équipements de transmission, en exploitant l'abstraction du réseau fourni par le plan de contrôle. Elles sont considérées davantage comme des API que comme protocole de programmation et de gestion de réseau. Il n'existe aucun standard intervenant entre la couche de contrôle et celle d'application. Selon l'ONF (Open Networking Foundation, 2012), plusieurs niveaux d'abstraction et différents cas d'utilisation peuvent être caractérisés, ce qui signifie qu'il peut y avoir plusieurs interfaces Nord pour servir tous les cas d'utilisation. Parmi les propositions des industriels, nous trouvons une API basée sur REST API (REpresentational State Transfer) pour

fournir une interface programmable utilisable par les applications. (Aichaoui, A., & Aitbelkacem, Y., 2018)

- **Interfaces Est/Ouest**

Un réseau d'entreprise à grande échelle est partitionné en sous-réseaux, où chaque sous-réseau est géré par un contrôleur différent. Dans cette architecture multi-contrôleur, chaque réseau a la vue globale du réseau pour communiquer avec l'autre domaine et échanger la topologie entre les sous-domaines. La communication intra-domaine directe n'est pas possible sans l'aide d'une interface appropriée. Les protocoles est-ouest résolvent ce problème en échangeant la vue du réseau entre les contrôleurs pour synchroniser l'état du réseau. (Sahoo, K. S., Mohanty, S., Tiwary, M., Mishra, B. K., & Sahoo, B., 2016).

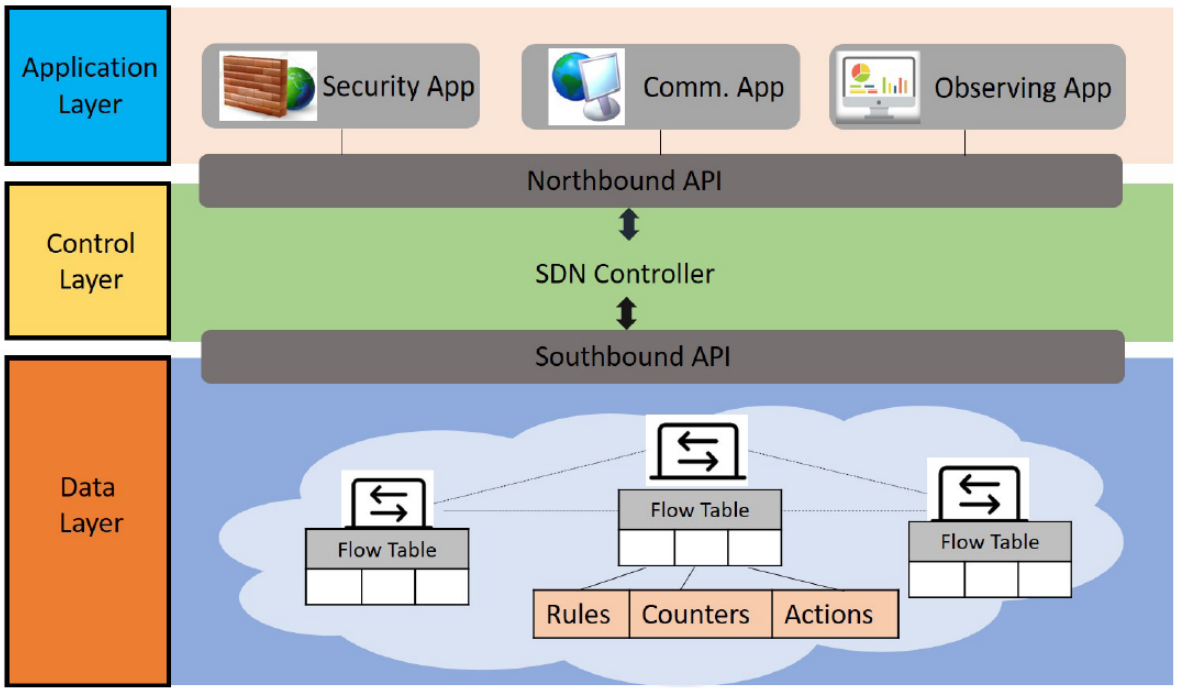


***Figure 2** : Les trois couches de l'architecture SDN*

La figure 3 situe les trois plans de l'architecture SDN et les interfaces qui les relient.

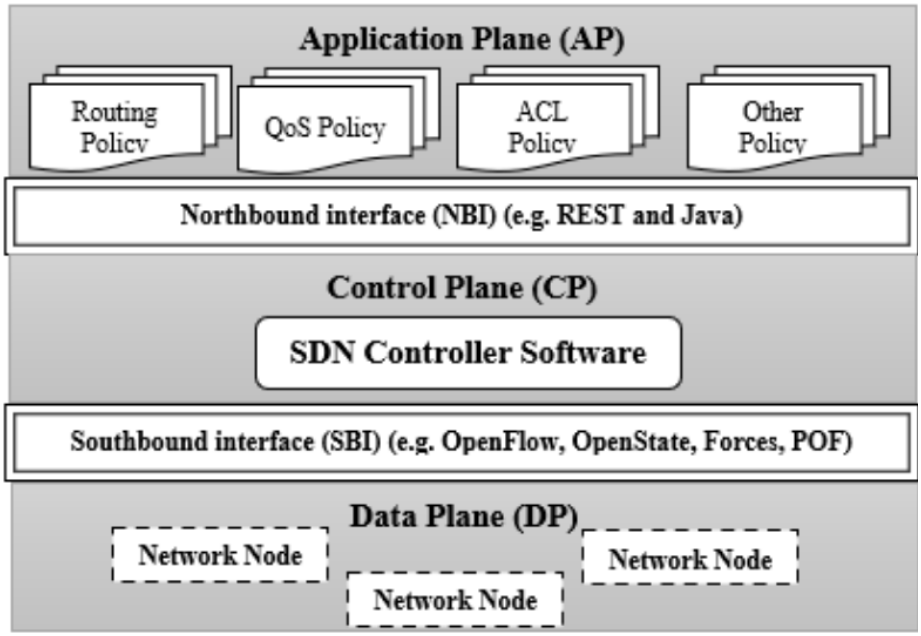


***Figure 3** : Plans et interfaces de l'architecture SDN*

3. LE PROTOCOLE OPENFLOW

Le protocole OpenFlow permet la séparation du plan de contrôle et du plan de données dans les réseaux en utilisant un contrôleur central, des tables de flux et une communication sécurisée. OpenFlow offre une flexibilité et une interopérabilité améliorées [4]. OpenFlow est jusqu'à nos jours la norme la plus populaire pour l'interface southbound des SDN. Il est standardisé par l'Open Networking Foundation (ONF) et implémenté par de nombreux équipementiers, dont HP (créateur avec Stanford university), Cisco (ACI format propriétaire), IBM, Juniper, NEC et Ericsson. Dans OpenFlow, les décisions de routage sont prises par le contrôleur pour chaque flux de données et poussées dans les switches sous forme de simples instructions de commutation.

L'architecture openflow est l'implémentation réelle des réseaux SDN, Cette architecture est basée principalement sur trois composantes : le plan de données, qui est composée des switches openflow ; le plan de contrôle, constitué par des contrôleurs OpenFlow ; une chaîne sécurisée qui permettent aux commutateurs de se connecter au plan de contrôle [3].

Pour définir un chemin de la source à la destination, OpenFlow offre un contrôle programmatique des flux aux administrateurs réseau. Il permet de supprimer la surcharge du traitement des paquets des routeurs pour définir le chemin, et les coûts de gestion du réseau lors

de l'escalade d'un réseau. Dans une architecture OpenFlow, l'ensemble du système se compose de nombreux dispositifs compatibles OpenFlow, qui sont gérés par un ou plusieurs contrôleurs OpenFlow. Les dispositifs OpenFlow utilisent une ou plusieurs tables de flux. Un flux pourrait être un ensemble de paquets avec les mêmes ID MAC ou adresses IP, des paquets avec la même balise VLAN ou des paquets arrivant du port [2].

La table de flux se compose d'entrées de flux, qui décident comment le paquet sera traité ou transféré pour un flux particulier. Chaque entrée de flux dans la table a trois champs : champ de correspondance (en-tête), champ d'action et champ de statistiques. Dans un commutateur OpenFlow, les paquets passent essentiellement par une série de tables de flux pour trouver leur correspondance exacte. Pour chaque paquet entrant, le commutateur prend la décision de transfert en consultant les entrées de la table de flux, en commençant par la première table et en se terminant soit par une correspondance dans l'une des tables, soit par un échec si aucune règle n'est présente pour le paquet. Une règle de flux peut être définie en combinant différents champs de correspondance, tels que le numéro de port, l'adresse IPv4 source ou de destination, l'adresse MAC source ou de destination, etc. S'il n'y a pas une telle entrée dans les tables de flux pour un flux, le paquet sera supprimé. Cependant, en cas de non-correspondance, le commutateur envoie le paquet correspondant au contrôleur. Parfois, le commutateur envoie le paquet non-correspondant au pipeline non-OpenFlow du commutateur [2].

4. LE CONTRÔLEUR SDN

C'est la partie principale du SDN qui fonctionne entre les dispositifs réseau et diverses applications qui fournissent une interface programmatique au réseau. Un contrôleur SDN a la seule responsabilité de gérer les protocoles réseau, les politiques et d'établir le chemin réseau d'un flux en installant des règles de flux sur les dispositifs réseau [2].

Le contrôleur SDN définit les flux de données entre le plan de contrôle centralisé et les plans de données sur des routeurs et des commutateurs individuels [5].

Pour pouvoir traverser le réseau, chaque flux doit être approuvé par le contrôleur SDN qui vérifie que la communication est autorisée dans le cadre de la politique réseau de l'entreprise. Si le contrôleur autorise le flux, il calcule l'itinéraire que ce dernier doit suivre et ajoute une entrée correspondante au flux dans tous les commutateurs situés sur le trajet [5].

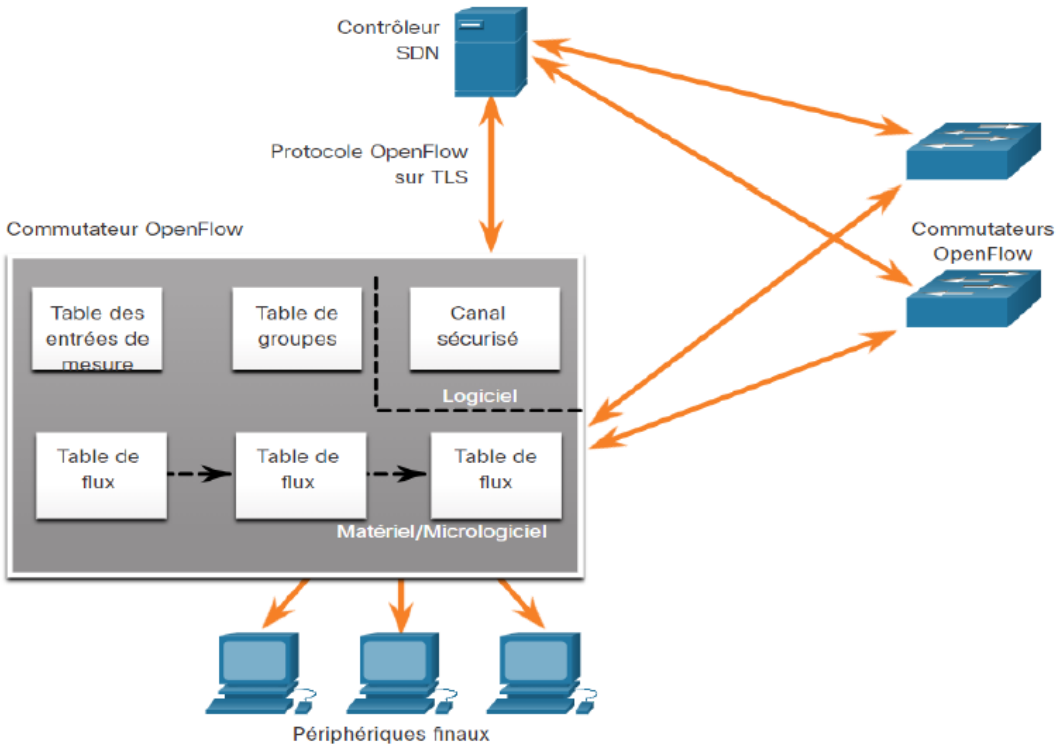


***Figure 4** : Fonctionnement du protocole OpenFlow*

Dans la figure 4, un contrôleur SDN communique avec les commutateurs prenant en charge **OpenFlow** à l'aide de ce protocole. Openflow s'appuie sur le protocole TLS (Transport Layer Security) pour sécuriser les communications issues du plan de contrôle à l'échelle du réseau. Chaque commutateur **OpenFlow** se connecte à d'autres commutateurs **OpenFlow**. Ils peuvent également se connecter aux périphériques utilisateur qui font partie d'un flux de paquets [5].

Sur chaque commutateur, la gestion des flux de paquets est assurée par une série de tables mises en œuvre au niveau du matériel ou du firmware. À l'échelle du commutateur, un flux est une séquence de paquets qui correspond à une entrée spécifique dans une table de flux [5].

5. LES TYPES DE SDN

Le premier point qui distingue une solution d'une autre est la façon dont les contrôleurs sont interconnectés les uns avec les autres [6].

- **Centralisée**

Cette architecture présente un seul contrôleur centralisé avec une vue globale du système. C'est l'architecture la plus simple et la plus facile à gérer, mais en même temps, la moins scalable/robuste en raison des problèmes bien connus des architectures centralisées (point de défaillance unique, goulots d'étranglement, partitionnement du réseau, etc.) [6].

- **Hiérarchique**

Architecture de type arbre composée de plusieurs couches de contrôleurs. La plupart des solutions présentent un arbre à deux niveaux consistant en des contrôleurs locaux et un contrôleur « racine ». Comme leurs noms l'indiquent, les contrôleurs locaux gèrent les opérations locales telles que le routage intra-site. À l'opposé, le contrôleur « racine » traite toutes les opérations inter-sites. Alors que les contrôleurs locaux n'ont que leur propre vue locale et ne sont pas conscients de l'existence des autres contrôleurs locaux, le contrôleur racine doit maintenir une connaissance globale de l'infrastructure pour communiquer avec les contrôleurs locaux chaque fois que cela est nécessaire. Bien que cette approche relève le défi de la scalabilité par rapport à l'approche centralisée, elle n'augmente la robustesse que partiellement, car le contrôleur racine reste un point centralisé [6].

- **Distribuée mais Logiquement Centralisée**

Architecture où il y a un contrôleur par site, gérant à la fois les opérations intra et inter-sites. Chaque fois qu'un contrôleur crée ou met à jour une ressource réseau, il diffuse les modifications à tous les autres contrôleurs. De cette manière, les contrôleurs peuvent maintenir une copie à jour de la connaissance globale, agissant ainsi comme une seule entité logique. Ce design se rapproche de la proposition initiale SDN, car plusieurs contrôleurs partagent des informations globales sur le réseau pour se présenter comme un seul contrôleur [6].

- **Totalement Distribuée**

Architecture similaire à la précédente mais sans communiquer toutes les créations/modifications aux autres contrôleurs. Dans cette approche, les données créées localement restent dans l'instance où elles ont été créées et ne sont partagées avec les autres instances que lorsque cela est nécessaire. Dans ce cas, des communications explicites entre les contrôleurs sont instanciées afin d'échanger des informations techniques pour établir, par exemple, des services inter-sites. Cette façon d'interconnecter les contrôleurs augmente la robustesse par rapport aux déconnexions réseau, car une déconnexion réseau ou une défaillance de nœud n'impacte qu'une sous-partie de l'infrastructure [6].

❖ **Hybride**

Architecture à deux couches mélangeant les architectures distribuée et hiérarchique. Le plan de contrôle consiste en plusieurs contrôleurs racine à la couche supérieure. Chacun des racines gère plusieurs contrôleurs locaux qui sont en charge de leurs sites respectifs. Ces contrôleurs racine sont organisés de manière distribuée, recueillant des informations sur l'état global du réseau parmi eux.

Les principales architectures du SDN sont représentées sur la figure 5.

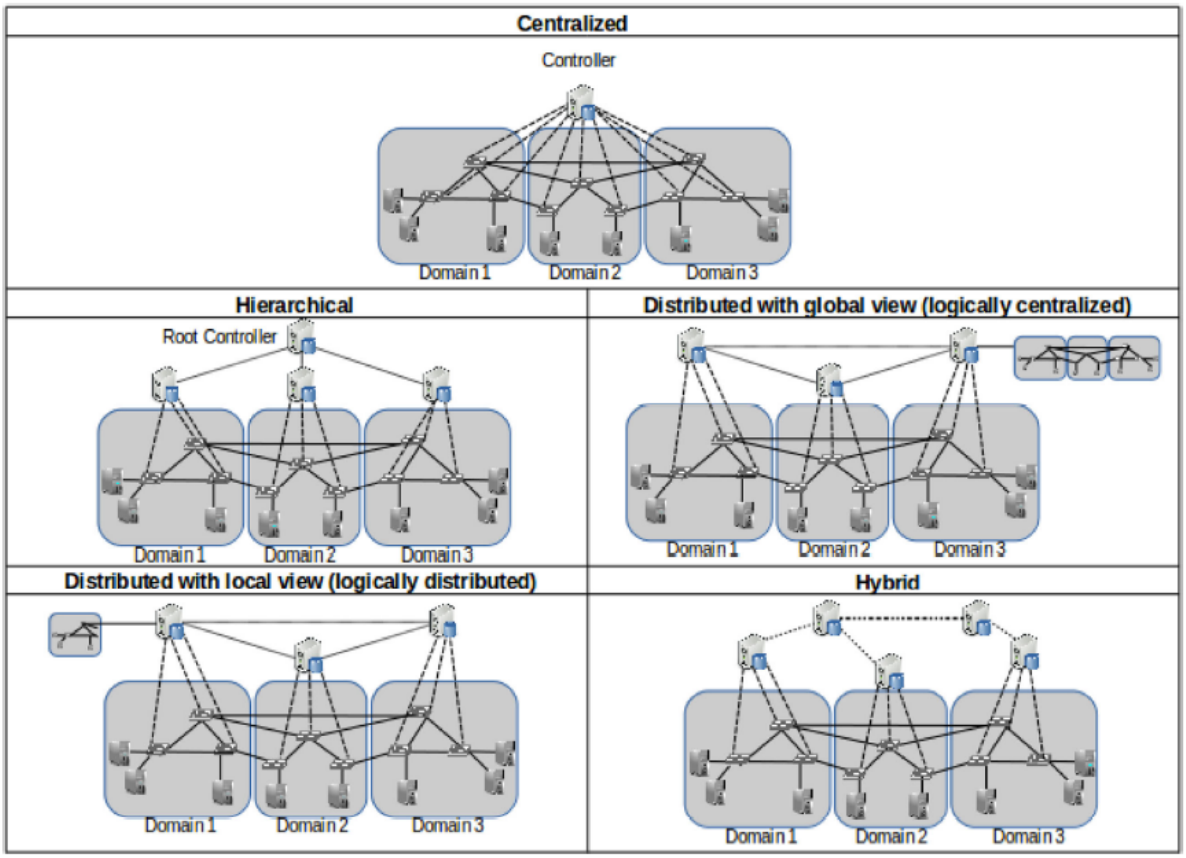


Figure 5 *: Architectures du SDN [6]*

6. Importance et avantages du SDN dans la modernisation des réseaux de télécommunication

L'adoption de SDN dans l'industrie des télécommunications a gagné un élan significatif en raison de sa capacité à relever les défis posés par les architectures réseau traditionnelles. SDN

offre une gamme d'avantages, notamment une agilité accrue, une meilleure visibilité réseau, une allocation dynamique des ressources et la capacité à déployer rapidement de nouveaux services et applications. En tirant parti de la nature programmable et logicielle de SDN, les opérateurs télécoms peuvent simplifier la gestion réseau, réduire les coûts opérationnels et améliorer l'expérience globale des utilisateurs.

- **Flexibilité et agilité réseau**

L'un des principaux avantages des réseaux définis par logiciel dans l'industrie des télécommunications est sa capacité à améliorer la flexibilité et l'agilité réseau. La séparation du plan de contrôle et du plan de données dans SDN permet une configuration et une gestion dynamiques des réseaux télécoms, permettant aux administrateurs réseau de répondre rapidement aux changements des demandes des utilisateurs, des schémas de trafic et des technologies émergentes. La flexibilité garantit que les réseaux télécoms peuvent s'adapter aux besoins divers des écosystèmes 5G et IoT, soutenant de nouveaux flux de revenus et améliorant l'expérience globale des utilisateurs[7].

- **Efficacité des coûts**

SDN offre également des opportunités significatives d'économies de coûts pour les opérateurs télécoms, tant en termes de dépenses en capital (CAPEX) que de dépenses opérationnelles (OPEX). La virtualisation du matériel réseau et le découplage des plans de contrôle et de données dans les réseaux basés sur SDN permettent aux opérateurs télécoms de réduire leur dépendance envers les équipements réseau propriétaires et spécifiques aux fournisseurs. Ce virage vers une architecture plus centrée sur le logiciel permet le déploiement de matériel commercial prêt à l'emploi, ce qui entraîne une réduction des CAPEX et une plus grande flexibilité dans la sélection et la mise à l'échelle du matériel. De plus, la nature centralisée et automatisée de la gestion réseau basée sur SDN réduit significativement les coûts opérationnels associés à l'administration réseau. SDN permet aux opérateurs télécoms de rationaliser les processus de configuration, de surveillance et de dépannage du réseau, réduisant le besoin d'interventions manuelles et abaissant les OPEX globaux [7].

- **Amélioration des performances réseau**

SDN joue un rôle crucial dans l'amélioration des performances et de l'efficacité des réseaux télécoms. Le plan de contrôle centralisé et les appareils réseau programmables dans les architectures basées sur SDN permettent une optimisation du trafic en temps réel et un

équilibrage de charge, garantissant que les ressources réseau sont utilisées efficacement et que les flux de trafic sont ajustés dynamiquement pour répondre aux demandes changeantes. Les réseaux télécoms compatibles SDN peuvent également offrir une qualité de service (QoS) améliorée et une gestion de la bande passante, permettant la priorisation des applications critiques et l'allocation efficace de la bande passante réseau. Cette amélioration des performances et de l'utilisation des ressources peut se traduire par de meilleures expériences utilisateur, une latence réduite et un débit plus élevé, tous essentiels dans la fourniture de services de télécommunication modernes [7].

- **Simplification de la gestion réseau**

L'adoption de SDN dans l'industrie des télécommunications simplifie le processus global de gestion réseau. Le plan de contrôle centralisé dans les architectures basées sur SDN offre aux administrateurs réseau une vue unifiée de l'ensemble du réseau, leur permettant de surveiller, dépanner et reconfigurer les éléments réseau plus efficacement. SDN soutient également la mise en œuvre de réseaux basés sur l'intention (intent-based networking), où les politiques et exigences réseau sont définies à un niveau supérieur, et le contrôleur SDN est responsable de traduire ces intentions en configurations réseau nécessaires. Cette approche favorise l'automatisation réseau, réduisant le risque d'erreurs manuelles et permettant aux opérateurs télécoms de répondre plus rapidement aux besoins commerciaux et opérationnels changeants [7].

II. GENERALITES SUR LE CLOUD EDGE COMPUTING

1. Paradigmes : Cloud, Fog, Edge, Mobile Edge Computing (MEC)

a. Le Cloud Computing

Selon **Amazon Web Services**, le cloud computing est la mise à disposition de ressources informatiques à la demande via Internet, avec une tarification à l'usage. Au lieu d'acheter, de posséder et de gérer des serveurs et des centres de données physiques, vous pouvez accéder à votre guise aux services technologiques, tels que la puissance de calcul, le stockage et les bases de données, d'un fournisseur cloud tel qu'Amazon Web Services (AWS) [8].

Le cloud computing concerne l'extension d'applications via internet en tant que services, ainsi que le logiciel et la technologie qui soutiennent les centres de données fournissant ces services. Le NIST formalise le cloud computing comme « un modèle pour permettre un accès réseau omniprésent, pratique, à la demande à un pool partagé de ressources de calcul configurables (par exemple, réseaux, serveurs, stockage, applications et services) qui peuvent être rapidement provisionnées et libérées avec un effort de gestion minimal ou une interaction avec le fournisseur de services ». Les caractéristiques essentielles du modèle cloud incluent le self-service à la demande, un accès réseau large, le pooling de ressources, une élasticité rapide et un service mesuré. Une infrastructure cloud englobe logiciel et matériel, étendant les fonctionnalités vitales du modèle cloud [14].

Les solutions cloud sont procurables via les modèles de service suivants [14] :

- **Software as a Service (SaaS)** : le fournisseur cloud présente aux consommateurs des applications accessibles via l'interface de programme ou un navigateur web, et le consommateur a un contrôle limité sur les applications spécifiques à l'utilisateur.
- **Platform as a Service (PaaS)** : Sur l'infrastructure cloud, les consommateurs sont autorisés à construire et distribuer des applications. Ils peuvent exercer un contrôle sur les applications déployées mais pas sur l'infrastructure cloud.
- **Infrastructure as a Service (IaaS)** : Le client est fourni avec des ressources de calcul essentielles vitales pour le traitement, le stockage et le réseautage. L'utilisateur exerce un contrôle sur le stockage, les applications et les systèmes d'exploitation, mais pas sur l'infrastructure cloud.

Les solutions cloud sont déployables comme [14] :

- **Cloud privé** : une organisation particulière qui peut aussi être un tiers possède et contrôle exclusivement le cloud privé, qui peut être disponible sur site ou hors site.
- **Cloud communautaire** : la communauté spécifique composée d'organisations qui partagent des préoccupations communes peut fournir une infrastructure cloud pour un

usage exclusif disponible sur site ou hors site, gérée par des organisations au sein de la communauté ou par un tiers.

- **Cloud public** : le cloud public disponible sur les locaux du fournisseur cloud peut être possédé ou géré par n'importe quelle entreprise et est ouvert à l'usage du grand public.
- **Cloud hybride** : il peut être composé de deux ou plus modèles cloud (privé, communautaire ou public) ; bien qu'ils soient des entités distinctes, ils sont liés par une technologie permettant la portabilité des applications ainsi que des données.

Les limites du Cloud Computing

Basé sur la croissance continue et massive du volume de données et les diverses exigences de traitement de données, le traitement de big data basé sur le cloud a montré de nombreux défauts [10]:

❖ Temps réel

Si un grand nombre d'appareils périphériques sont ajoutés et qu'un grand volume de données terminales est toujours transmis au cloud pour traitement, le volume de transmission de données intermédiaires sera considérablement augmenté, les performances de transmission de données seront réduites, entraînant une charge importante de la bande passante de transmission réseau, provoquant un délai de transmission de données. Dans certains scénarios d'application nécessitant un retour en temps réel, tels que le trafic, la surveillance, etc., le Cloud Computing ne pourra pas répondre aux exigences commerciales en temps réel.

❖ Sécurité et confidentialité

Par exemple, lors de l'utilisation de diverses applications dans les smartphones, les applications nécessiteront des données utilisateur, y compris des données privées. Il existe un risque élevé de fuite de confidentialité ou d'attaque sur ces données lorsqu'elles sont téléchargées vers le centre cloud.

❖ Consommation d'énergie

Le nombre d'appareils intelligents continue d'augmenter, et la consommation d'énergie des centres de données en Chine a augmenté de manière significative. L'amélioration de l'efficacité d'utilisation de la consommation d'énergie du Cloud Computing ne peut pas répondre à la demande croissante de consommation d'énergie des données. La société intelligente en développement rapide aura des exigences plus élevées en matière de consommation d'énergie du Cloud Computing.

En raison de l'augmentation du volume de données et des exigences croissantes en matière de traitement de données, l'Edge Computing a émergé au moment historique.

b. L'Edge Computing

LEdge Computing (EC) ou l'informatique de périphérie est un paradigme informatique distribué qui rapproche les ressources informatiques de la périphérie du réseau, là où les données sont générées et consommées, plutôt que dans des centres de données centralisés. Dans l'informatique de périphérie, les services de traitement, de stockage et de mise en réseau des données sont fournis par des nœuds situés à la périphérie du réseau, tels que des routeurs, des passerelles et des appareils IoT, plutôt que par des serveurs centralisés [11].

La croissance explosive et l'augmentation de la puissance de calcul des appareils IoT ont généré des volumes de données sans précédent, qui continue de croître à mesure que les réseaux 5G augmentent le nombre d'appareils mobiles connectés. L'envoi de toutes les données générées par ces appareils vers un centre de données centralisé ou vers le cloud pose des problèmes de bande passante et de latence. L'Edge Computing offre une alternative plus efficace : les données sont traitées et analysées plus près du point où elles sont créées. Ainsi, les données ne transitent pas par un réseau jusqu'au cloud ou jusqu'à un centre de données pour être traitées, ce qui réduit le temps de latence. L'Edge Computing permet une analyse plus rapide et plus complète des données, offrant ainsi la possibilité d'approfondir les informations, d'accélérer les temps de réponse et d'améliorer l'expérience utilisateur [11].

L'objective principale de ce paradigme est de réduire les paramètres de réseau à savoir, la latence et l'utilisation massive de la bande passante. En termes plus générale, les applications Edge Computing consiste à exécuter moins de processus dans le Cloud distant en déplaçant ces processus vers des environnements locaux, tels que les dispositifs d'un utilisateur, un appareil PDA ou un serveur de périphérie. De plus, amener le calcul à la périphérie du réseau minimise la quantité de communication longue distance qui doit se produire entre un client et un serveur [12].

Dans les architectures réseau cellulaire, les serveurs sont déployés à proximité de chaque station de base (BS). Les utilisateurs utilisent alors les ressources fournies par les serveurs situés dans la cellule à laquelle ils sont connectés. Ces serveurs effectuent les calculs nécessitant un temps de réponse faibles et en transmettent au Cloud ceux qui nécessitent des ressources plus conséquentes. Une telle architecture est également utile lors du stockage de données et le

passage rapide des demandes de l'utilisateur au serveur situé en périphérie du réseau. Si ce réseau ne dispose pas la ressource demandée, la requête est transmise à l'infrastructure Cloud. Lorsque les données sont renvoyées par le Cloud, elles sont mises en cache afin de pouvoir répondre directement aux futures requêtes. Les serveurs situés à la périphérie du réseau ont des ressources moins importantes que celles du Cloud mais suffisantes pour effectuer certains calculs ayant besoin d'une faible latence [12].

c. Le Fog Computing

Le Fog Computing est fréquemment délibéré comme une forme de Edge Computing. Le Fog computing livre littéralement un traitement distribué, un réseau et un potentiel de stockage plus proche de l'utilisateur. Le Fog Computing est plus qu'un simple déploiement du concept de Edge Computing ; c'est le sommet du renforcement des concepts de Edge Computing. Ce n'est pas une extension ou un remplacement du Cloud ; c'est un nouveau paradigme opérant entre l'IoT et le cloud, avec l'intention de soutenir et d'améliorer l'interaction et l'intégration du cloud, de l'edge et de l'IoT [14].

Cisco décrit le Fog Computing comme un ensemble hautement virtualisé qui fournit des services de calcul, de stockage et de réseau entre les centres de données cloud conventionnels et les dispositifs finaux, généralement mais pas exclusivement déployés au bord du réseau. Selon l'OpenFog Consortium, le fog computing est défini comme « une architecture horizontale de niveau système qui distribue des ressources et des services de calcul, de stockage, de contrôle et de réseau n'importe où le long du continuum du Cloud aux Objets, accélérant ainsi la vélocité de la prise de décision ». Le fog computing adopte une tactique distribuée originaire du modèle de edge computing pour surpasser les limitations de l'approche de cloud computing centralisée, avec des nœuds fog positionnés n'importe où entre le cloud et les dispositifs finaux [14].

d. Le Mobile Edge Computing (MEC)

Le MEC est une continuation du calcul mobile via l'edge computing, offrant le calcul et le stockage aux dispositifs mobiles contraints en énergie et ressources [14].

Avec le MEC, un serveur cloud est déployé aux stations de base du réseau cellulaire. Il exécute des tâches telles que l'amélioration de la performance de l'application et la minimisation de la congestion réseau, de l'utilisation de la bande passante et de la latence pour les abonnés qui ne peuvent pas être atteints avec une architecture réseau conventionnelle. Même si les capacités de traitement et de mémoire des dispositifs mobiles s'améliorent, ils ne sont toujours pas

suffisants pour gérer des tâches intensives en calcul, ce qui a conduit aux modèles MCC (Mobile Cloud Computing) et MEC. Le MEC réduit considérablement la durée du processus et les exigences énergétiques des dispositifs mobiles en installant des ressources computationnelles proches des stations de base. Comme les stations de base servent de points d'accès cruciaux à plusieurs dispositifs IoT, les dispositifs finaux pourraient être directement servis avec un seul saut via le MEC. Le MEC met en valeur une faible latence, une proximité à l'utilisateur, une conscience de la localisation et une distribution géographique. Cependant, il restreint le besoin d'installer un serveur MEC dédié pour les services MEC. Avec l'augmentation de la demande de ressources au fil du temps, l'évolutivité est un autre défi majeur. Le MEC présente la fiabilité, l'efficacité énergétique et une faible latence adaptées à diverses applications et surpasse le MCC de manière comparable [14].

2. Architecture et caractéristiques du Cloud Edge

La figure 6 illustre le déchargement des traitements vers le serveur Edge.

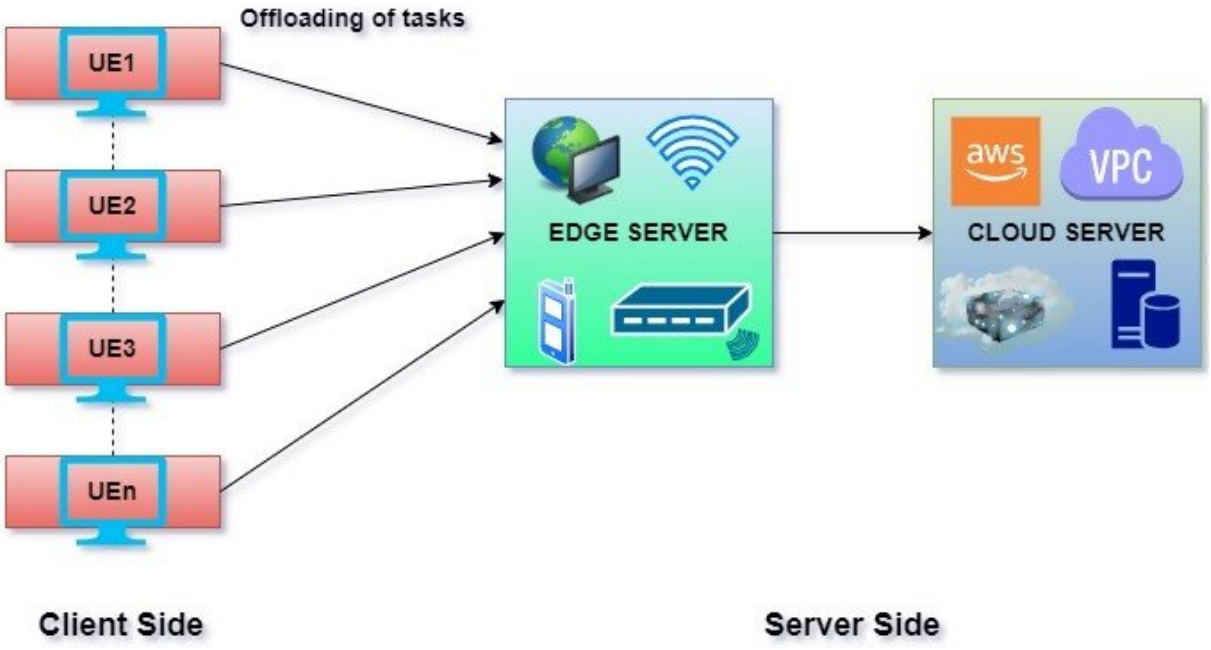


***Figure 6** : Déchargement des traitements vers le serveur Edge*

L'Edge Computing se compose de trois couches principales : la couche cloud, la couche Edge et la couche dispositifs [15].

La couche supérieure est la couche cloud, qui peut être une infrastructure cloud publique ou privée. Cette couche héberge et exécute les applications gourmandes en calcul que la couche Edge ne peut pas prendre en charge. De plus, elle exécute également les applications chargées d'orchestrer et de gérer les différents nœuds Edge. La charge de travail provenant de la couche Edge sera déchargée vers le cloud et interagira avec lui. Le cloud peut également servir de source principale de stockage selon les besoins.

La deuxième couche est la couche Edge. Cette couche se décline en quatre formes d'implémentation courantes : le nœud Edge, le Cloudlet, le Fog Computing et le Mobile Edge Computing (MEC). Ces formes diffèrent toutefois par leur emplacement, leurs fonctionnalités et leur architecture. Les nœuds Edge désignent tout serveur ou passerelle Edge où la fonction EC est réalisée. Ces nœuds peuvent être subdivisés en partitions virtuelles selon les services fournis, par exemple, des nœuds Edge dédiés aux infrastructures de villes intelligentes dans une première partition, des nœuds Edge dédiés aux établissements médicaux dans une autre partition, et ainsi de suite. Chaque partition virtuelle communique avec sa couche de dispositifs respective.

Un Cloudlet fonctionne de manière similaire à un Cloud conventionnel en offrant des services de calcul, mais à une échelle réduite et avec des ressources limitées par rapport à un cloud traditionnel. Les utilisateurs sont directement connectés aux autres dispositifs au sein d'un cloudlet spécifique, souvent localisé dans une région particulière. Cette spécificité régionale est fréquemment observée dans les dispositifs mobiles, réduisant considérablement la latence des réponses aux requêtes des utilisateurs. De plus, lorsqu'un dispositif mobile se déplace d'une région à une autre, il bascule d'un cloudlet à un autre. Les cloudlets peuvent ainsi être utilisés dans des environnements nécessitant une excellente conscience situationnelle, une prise de décision efficace et une connectivité fiable.

Le MEC correspond à l'EC appliqué aux dispositifs mobiles. Son objectif principal est de réduire la latence et la consommation énergétique des appareils mobiles en leur permettant de délester leurs tâches intensives en calcul vers des serveurs de proximité. Cela est rendu possible grâce à l'intégration de ressources de calcul, de stockage et de réseau avec les stations de base 3G, LTE et 5G.

La troisième couche est la couche dispositifs, qui regroupe les équipements Edge tels que les capteurs, les caméras et les contrôleurs. Ces équipements peuvent collecter et

transmettre des données vers la couche Edge. Cependant, cette capacité reste limitée en raison de la taille réduite des dispositifs, ce qui contraint leurs capacités de calcul et de stockage.

L'organisation générale de l'Edge Computing est représentée sur la figure 7.

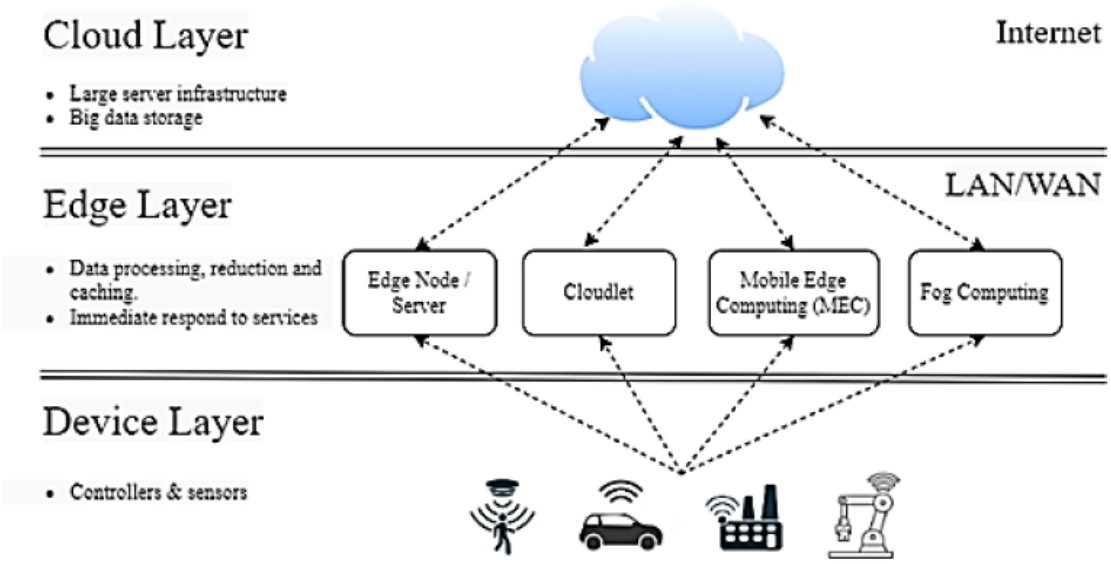


Figure 7 *: Architecture de l'Edge Computing [15]*

3. Les avantages du Edge Computing

L'Edge Computing offre plusieurs avantages par rapport aux modèles informatiques centralisés traditionnels à savoir : [10]

- **Latence réduite** : en traitant les données plus près de la périphérie du réseau, l'Edge Computing peut réduire la latence associée à l'envoi de données vers des centres de données centralisés.
- **Fiabilité améliorée** : l'Edge Computing peut améliorer la fiabilité des applications en réduisant la dépendance vis-à-vis des centres de données centralisés et des connexions réseau qui les relient.
- **Besoins réduits en bande passante** : en traitant les données à la périphérie du réseau, l'Edge Computing peut réduire la quantité de données qui doit être envoyée aux centres

de données centralisés pour le traitement, ce qui peut réduire les besoins en bande passante.

- **Améliorer la sécurité des données** : l'Edge Computing permet d'améliorer la sécurité des applications en réduisant la surface d'attaque associée aux centres de données centralisés et aux connexions réseau qui les relient.

4. Cas d'usage de l'Edge Computing

a. La santé

La quantité de données générées dans le domaine de la santé a considérablement augmenté au cours des dernières années, notamment grâce aux appareils connectés et à l'essor des soins à distance. De plus, dans le contexte sanitaire actuel, la supply-chain pharmaceutique et notamment des vaccins nécessite un suivi précis et un conditionnement sans faille. L'Edge Computing peut contribuer fortement à l'amélioration du traitement des données médicales et du transport pharmaceutique [16].

➢ Supply chain pharmaceutique

Avec des composants provenant du monde entier, il est essentiel que la supply chain pharmaceutique soit sécurisée et traçable La protection de la qualité et de la sécurité des composants est une grande préoccupation pour de nombreuses marques pharmaceutiques tout au long de la chaîne d'approvisionnement. La stabilité de la température est importante tout au long de la chaîne d'approvisionnement. De nombreux médicaments biologiques fabriqués à partir d'organismes vivants et vaccins nécessitent des plages de température spécifiques pendant le transport et le stockage. Si la température fluctue en dehors de la plage requise, un lot entier pourrait être compromis. L'Edge computing peut ainsi aider à garantir que la température soit surveillée et contrôlée en permanence pendant l'expédition et le stockage, avec une surveillance localisée via des capteurs et des appareils IoT dans les véhicules de transport et les entrepôts permettant la transmission en temps réel d'informations[16].

➢ Capteurs médicaux

De nombreux dispositifs médicaux IoT tels que des capteurs portables, des glucomètres et des applications de soins de santé sont devenus beaucoup plus courants au cours de la dernière décennie, collectant des quantités massives de données de santé générées par les patients qui

permettent aux professionnels de la santé de mieux diagnostiquer les problèmes, et surveiller la santé de leurs patients sur de longues périodes. La quantité massive de données produites par ces appareils peut être précieuse, mais elle crée également un défi pour les professionnels de santé chargés de la gérer et de la maintenir en sécurité. Une grande partie de ces données n'est pas structurée et reste mal définie, inondant des infrastructures Cloud qui ne sont souvent pas prêtes à exécuter les puissants programmes d'analyse nécessaires pour les organiser de manière à être facilement utilisées. Au moment où les données générées par les appareils de périphérie IoT sont renvoyées dans un serveur central pour être correctement analysées et triées, il peut être trop tard pour répondre aux changements soudains de l'état d'une personne. L'Edge computing a le potentiel de résoudre ce problème de données. En conservant une grande partie des tâches de traitement critiques à la périphérie du réseau, les architectures informatiques des services de santé peuvent toujours bénéficier de la collecte de données tout en obtenant des analyses en temps réel qui peuvent prédire et répondre à une urgence [16].

- **Diagnostic médical avancé**

Les dispositifs d'imagerie médicale, tels que les scanners IRM et CT, pourront générer de grandes quantités de données. L'Edge Computing peut être utilisé pour traiter ces données localement, réduisant ainsi la latence et évitant le transfert massif de données vers des serveurs distants. Les capteurs portables et les dispositifs médicaux IoT une fois implémentés dans un hôpital peuvent bénéficier du Edge Computing en effectuant un traitement préliminaire des données localement, avant de transmettre uniquement les informations pertinentes aux systèmes centraux. Cela permet une surveillance médicale continue et en temps réel des patients, facilitant le diagnostic précoce des conditions médicales et la gestion des maladies chroniques. Les algorithmes d'IA intégrés au Edge Computing peuvent aider à analyser les images médicales en temps réel pour un diagnostic plus rapide et plus précis [17].

b. Véhicule autonome

Le véhicule autonome représente l'objet connecté par excellence. Il est en communication constante avec son environnement et prend des décisions rapidement en fonction des informations provenant d'un ensemble de capteurs. En particulier, la communication machine-to-machine entre une voiture autonome et les autres véhicules sur la route est primordiale. Chaque véhicule autonome capte et diffuse des données critiques (ex : distance, vitesse,

signalisation …) permettant aux autres véhicules de se renseigner suffisamment tôt sur les dangers tels que les détours, les débris, les accidents et les inondations, pour s'adapter en conséquence. Une grande partie de ces données pourront être envoyées et reçues entre les véhicules eux-mêmes, sans les obliger à s'interfacer avec des serveurs Cloud distants. Cette communication transforme chaque véhicule en circulation en une extension des capteurs de tous les autres véhicules, fournissant ainsi des informations pertinentes sur l'environnement routier. En raison du grand volume de données produites et reçues de l'environnement, L'Edge computing permet aux véhicules d'avoir un accès et une analyse directe des données, évitant ainsi une surcharge d'informations non critiques dans le Cloud, mais également de réduire les temps de latence liés à l'échange de données avec des serveurs Cloud distants. En déployant des installations Edge à la fois dans des zones à fort trafic et dans des zones plus éloignées avec un accès limité à la bande passante, les entreprises peuvent garantir que leurs véhicules autonomes peuvent accéder aux données dont ils ont besoin avec une latence minimale pour prendre des décisions rapidement [16].

c. Distribution

Le monde de la distribution est en train de changer radicalement En utilisant massivement les données récoltées sur les clients, les distributeurs sont en mesure d'en apprendre beaucoup plus sur leurs habitudes, leurs comportements d'achats et sur la rapidité avec laquelle ils changent leur façon de consommer des produits En effet, de nos jours, les clients sont en constante recherche de la meilleure combinaison entre achat en ligne et « In-Store » Par conséquent, les futurs leaders du secteur du Retail dans les prochaines années seront ceux qui maitriseront cette combinaison et qui analyseront les données clients suffisamment rapidement pour répondre à leurs attentes L'Edge Computing peut les aider à répondre à ces problématiques [16].

➢ Gestion des stocks

Une gestion efficace des stocks est un défi majeur pour les entreprises de logistique ou de la distribution. Le suivi des stocks en magasin permet aux entreprises de prendre des décisions plus éclairées et d'éviter les ruptures d'approvisionnement, notamment en période de forte affluence. Pour faire face à cette augmentation de la demande, les retailers doivent surveiller en temps réel l'état du stock dans les magasins et savoir où se trouvent les produits dans la chaîne

d'approvisionnement à tout moment. Pour y parvenir efficacement, une entreprise doit collecter de grandes quantités de données sur la position et l'état des marchandises en temps réel, in-store et en acheminement. Une infrastructure de Edge pour la gestion des stocks en magasin permet de suivre le stock existant en temps réel de façon continue grâce aux faibles temps de latence, et d'automatiser le renouvellement du stock en prévision d'éventuelles ruptures grâce à des algorithmes de Machine Learning. Ce type d'application permet de limiter le temps alloué à la gestion des stocks et de garantir une disponibilité continue des produits. Couplée à une infrastructure Edge dans la Supply Chain, notamment au sein des transporteurs de produits, il est possible de suivre en temps réel l'acheminement de la marchandise et de gérer les stocks en magasin avec une grande précision [16].

➢ Suivi du parcours client en point de vente

Les magasins de la grande distribution utilisent de plus en plus des interfaces numériques interactives afin d'améliorer l'expérience des clients, à la fois pour encourager les clients à passer plus de temps en magasin, mais aussi pour inciter les consommateurs à faire leurs achats en magasin plutôt qu'en ligne [16].

L'Edge computing peut être utilisé pour exécuter certaines applications au sein des magasins qui nécessitent une interaction humaine en temps réel, comme un miroir de réalité augmentée dans chaque cabine d'essayage, permettant au client d'essayer un vêtement sans l'enfiler, changer de couleur, essayer une autre taille et vérifier la disponibilité des stocks en temps réel. Via l'utilisation d'un caddy connecté, en fonction de la disponibilité des articles en rayon, le client pourra recevoir des notifications pour l'informer de la disponibilité d'un article et se voir suggérer des articles en se basant sur les produits sélectionnés pendant son parcours. Cette personnalisation du parcours client est possible grâce à un échange de données en temps réel, apportant ainsi de la valeur aux clients en leur offrant un endroit agréable et pratique pour faire des achats. Ces interactions avec le client permettront également d'évaluer l'affluence au sein du magasin, et d'adapter en temps réel l'organisation des conseillers de vente afin de répondre à la demande de clients au niveau des rayons et des caisses, notamment grâce à des algorithmes de prédiction basés sur le Machine Learning [16].

III. LE SDN DECENTRALISE ET LE CLOUD EDGE

IV. L'OPTIMISATION

L'optimisation réseau désigne l'ensemble des techniques et stratégies utilisées pour améliorer l'efficacité, la performance et la capacité d'un réseau informatique. Elle peut inclure l'amélioration de la gestion de la bande passante, la réduction de la latence et l'optimisation du routage des données. Ces pratiques sont essentielles pour garantir une connexion rapide et fiable, particulièrement dans un contexte où la demande en données ne cesse de croître [18].

1. Les approches classiques d'optimisation et ses limites

Le Software-Defined Networking (SDN) a révolutionné l'architecture des réseaux en séparant le plan de contrôle du plan de données, offrant ainsi une programmabilité centralisée, une visibilité globale et une flexibilité accrue (Firouzi & Rahmani, 2022 ; Amin et al., 2024) [27] [28] . Cependant, malgré ces avantages, les contrôleurs SDN traditionnels (comme POX, ONOS ou Ryu) rencontrent des limites importantes face à la dynamique du trafic, à la congestion, à la scalabilité et aux exigences de qualité de service (QoS) dans les environnements modernes (IoT, Edge Computing, 5G/6G) (Senthilkumaran & Sankarasubramaninan, 2025 ; Ali & Awad, 2025) [29] [30].

Les algorithmes de routage classiques (shortest-path, OSPF, ECMP) sont statiques et ne s'adaptent pas efficacement aux variations du trafic, entraînant une sous-utilisation des ressources et une dégradation des performances (Wang et al., 2024 ; Hamzah, 2025) [31] [25].

2. Apport de l'apprentissage automatique (Machine Learning)

Les avancées récentes en matière d'optimisation réseau sont principalement influencées par le développement technologique, y compris l'intelligence artificielle (IA), l'apprentissage automatique, et les technologies de big data. Ces innovations permettent à l'optimisation réseau d'aborder des problèmes de plus grande échelle et plus complexes avec une précision et une efficacité accrues [18].

L'apprentissage automatique est un domaine d'étude de l'intelligence artificielle qui se base sur le développement d'une capacité d'apprentissage des ordinateurs à partir d'approche statistiques de données. Il permet à des ordinateurs d'apprendre à effectuer plus rapidement des tâches normalement réalisées par l'homme.

Le machine learning repose sur deux piliers fondamentaux [20]:

- **les données**, qui sont les exemples à partir duquel l'algorithme va apprendre ;
- **l'algorithme d'apprentissage**, qui est la procédure que l'on fait tourner sur ces données pour produire un modèle. On appelle entraînement le fait de faire tourner un algorithme d'apprentissage sur un jeu de données.

Ces deux piliers sont aussi importants l'un que l'autre. D'une part, aucun algorithme d'apprentissage ne pourra créer un bon modèle à partir de données qui ne sont pas pertinentes : c'est le concept garbage in, garbage out qui stipule qu'un algorithme d'apprentissage auquel on fournit des données de mauvaise qualité ne pourra rien en faire d'autre que des prédictions de mauvaise qualité. D'autre part, un modèle appris avec un algorithme inadapté sur des données pertinentes ne pourra pas être de bonne qualité [20].

Dans l'apprentissage automatique, il est nécessaire de distinguer les notions d'apprentissage automatique et d'apprentissage profond, qui représentent différentes branches de l'IA [19]

a- L'apprentissage Automatique

L'apprentissage automatique, ou Machine Learning représente la majorité des algorithmes de classification, de régression et d'ordonnancement, avec un fonctionnement simple. En

apprentissage automatique, deux grandes catégories coexistent, les algorithmes supervisés, non-supervisés et l'apprentissage par renforcement.

❖ Les algorithmes supervisés

Ils sont utilisés pour résoudre des problèmes de régression, classification ou ordonnancement. Ces algorithmes nécessitent un jeu d'entraînement préalable à partir duquel un modèle d'apprentissage est créé. Ce modèle d'apprentissage est ensuite appliqué sur un jeu de données pour réaliser des prédictions [19]. Les algorithmes supervisés se répartissent sur 3 principales familles :

- ***Les Modèles linéaires***

D'un point de vue purement mathématique, les modèles linéaires sont les algorithmes les plus simples et facilement compréhensibles. Ils se basent principalement sur quelques principes tels que : le suivi de tendances, le voisinage ou la séparation en groupe dans des hyperplans. Dans cette classe d'algorithme, nous avons entre autres : K plus proches voisins, Séparateur à Vaste Marge, Bayesien gaussien

- ***Les Arbres de décisions***

Les arbres de décision fonctionnent par une suite de découpes optimisées du jeu d'entraînement selon un paramètre aléatoire. Le seuil de découpe utilisé par les arbres de décisions n'est pas aléatoire, mais vise à réduire l'entropie au minimum possible. Chaque élément découpé étant ensuite redécoupé individuellement, un arbre est obtenu en grandissant jusqu'à atteindre des branches ne contenant qu'une seule classe ou une limite prédéfinit. Dans cette classe, nous avons les algorithmes de Forêts aléatoires et Renforcement de gradient

- ***Les Réseaux neuronaux***

Un réseau de neurones est un modèle informatique qui ressemble à la structure en réseau des neurones du cerveau. Il est composé d'éléments de traitement interconnectés, appelés neurones, qui travaillent en synergie pour produire une fonction de sortie. Les réseaux de neurones sont structurés en couches ou en dimensions d'entrée et de sortie. Dans la plupart des cas, ils comportent également une couche cachée composée d'unités qui transforment l'entrée afin qu'elle puisse être exploitée par la couche de sortie [23].

Ce simule le fonctionnement du cerveau humain, avec un grand nombre de nœuds de traitement reliés qui facilitent des processus comme la traduction en langage naturel, la reconnaissance d'images, la reconnaissance vocale et la création d'images.

- **Les algorithmes Non Supervisé** :

Ici, l'algorithme est entraîné sur des données non-étiquetées. L'objectif est de découvrir des structures cachées dans les données. Sans réponses fournies, l'algorithme tente de regrouper ou de réduire la dimensionnalité des données afin d'extraire des informations potentiellement pertinentes [22]

Ces algorithmes sont [21]

- **Clustering (regroupement)**

Cette méthode est utilisée lorsque vous devez segmenter un ensemble de données en groupes distincts basés sur des caractéristiques similaires. Par exemple, vous pouvez utiliser la méthode K-means pour *segmenter une clientèle à des fins de marketing ciblé.*

- **Réduction de dimensionnalité**

Cette technique peut être utilisée pour réduire la dimensionnalité lors du traitement de données multivariées. Elle permet de visualiser des données de grande dimension et est utile dans des domaines tels que *l'analyse de l'expression génétique ou la segmentation de la clientèle.*

- **Regroupement hiérarchique**

Similaire au K-means, cet algorithme permet également de segmenter les données. La différence réside dans le fait que le regroupement hiérarchique crée un modèle arborescent des données, permettant de visualiser les regroupements imbriqués. Cette méthode peut être utilisée pour *catégoriser des documents écrits en fonction de leur sujet.*

- **Apprentissage par Renforcement :**

c'est une approche de l'apprentissage automatique où un agent apprend à prendre des décisions dans un environnement dynamique pour maximiser une récompense cumulative. Contrairement à l'apprentissage supervisé et non supervisé, l'apprentissage par renforcement repose sur l'interaction continue de l'agent avec l'environnement, recevant des retours sous forme de récompenses ou de pénalités en fonction des actions entreprises. L'agent ajuste sa

stratégie au fil du temps pour améliorer ses performances, apprenant quelle action entreprendre dans différentes situations pour obtenir la plus grande récompense possible. Nous pouvons citer [22] :

- **Algorithmes Basés sur la Valeur (*Value-Based Algorithms*) :**

Ces algorithmes cherchent à apprendre une fonction qui estime la "valeur" attendue (la récompense future) de prendre une action donnée dans un état donné. L'agent choisit ensuite l'action qui a la plus grande valeur estimée. Parmi ces algorithmes, nous pouvons citer : *Q-Learning, Deep Q-Network.*

- **Algorithmes Basés sur la Politique (*Policy-Based Algorithms*)**

Contrairement aux algorithmes basés sur la valeur, ceux-ci apprennent directement une "politique" (une fonction qui mappe un état à une action) sans passer par l'estimation de la valeur. L'objectif est d'ajuster les paramètres de la politique pour maximiser la récompense. On peut citer : *REINFORCE, Policy Gradient (PG).*

L'intégration de l'IA dans les SDN est prometteuse pour aider au problème d'équilibrage de charge. L'IA fournit plus d'intelligence et de capacités de gestion de réseau automatisées. Des études précédentes ont mis en évidence les efforts en cours pour intégrer l'IA dans les SDN [56]. L'application de l'intelligence artificielle aux réseaux définis par logiciel révolutionne la gestion du réseau en infusant des capacités de prise de décision intelligente dans l'infrastructure réseau. Grâce à l'IA, les SDN peuvent s'adapter dynamiquement aux conditions changeantes du réseau, optimiser le routage du trafic et renforcer les protocoles de sécurité en temps réel. Les algorithmes d'apprentissage automatique analysent de vastes quantités de données réseau pour identifier des motifs, prédire le comportement du réseau et atténuer proactivement les problèmes potentiels. [no binding architechture]

V. PROBLEMATIQUE

Au cours de la dernière décennie, l'Internet des objets (IoT) a connu un changement de paradigme, passant d'un modèle de calcul centralisé dans le cloud à un modèle basé sur le calcul en périphérie (Edge Computing). Des centaines de milliards d'objets devraient être déployés

dans ce paradigme IoT en pleine expansion, générant ainsi d'énormes volumes de données. La transmission de toutes ces données vers le cloud s'est récemment révélée être un goulot d'étranglement en termes de performance, car elle entraîne de nombreux problèmes liés au réseau, notamment la latence, la consommation d'énergie, la sécurité et la confidentialité [24].

En raison du dynamisme qui caractérise le développement des technologies de mise en réseau, un niveau supérieur de gestion réseau a émergé, connu sous le nom de réseau défini par logiciel (SDN). Cependant, des études ont également mis en évidence les difficultés suivantes pour obtenir des résultats optimaux dans les environnements SDN : la tâche devient particulièrement complexe dans les scénarios dynamiques qui exigent souvent un contrôle efficace des flux de trafic et une allocation efficiente des ressources dans des environnements en temps réel. Les mécanismes conventionnels distribués de gestion du trafic se révèlent fréquemment incapables de faire face à une distribution imprévisible du trafic et à une complexité réseau croissante [25].

Ces défis peuvent notamment être surmontés en exploitant la puissance de l'apprentissage automatique (ML), qui intègre des prises de décision intelligentes en temps réel et des capacités prédictives dans les environnements SDN. Certaines études récentes ont démontré comment l'utilisation pratique de techniques basées sur le ML peut améliorer les flux de trafic, minimiser la congestion et accroître la qualité de service (QoS) [25].

Question de recherche :

Comment optimiser dynamiquement une architecture SDN décentralisée dans un environnement Cloud-Edge distribué en utilisant des techniques de Machine Learning afin d'améliorer les performances réseau ?

Objectifs :

Objectif général

Concevoir et évaluer un système d'optimisation dynamique d'une architecture SDN décentralisée dans un environnement Cloud-Edge distribué, en exploitant des techniques de Machine Learning pour améliorer les performances réseau.

Objectifs spécifiques

1. Analyser les limites des architectures SDN existantes en environnement Cloud-Edge

- Identifier les problèmes de **déséquilibre de charge** entre les contrôleurs distribués
- Étudier les **contraintes de latence** entre les nœuds Edge et le Cloud
- Recenser les **inefficacités des mécanismes statiques** de placement et d'association des contrôleurs
- Évaluer l'impact de la **mobilité des nœuds** sur la stabilité de l'architecture SDN

2. Modéliser l'architecture SDN décentralisée Cloud-Edge

- Définir une **architecture hiérarchique** à plusieurs niveaux (Edge local, Edge régional, Cloud central)
- Concevoir un **modèle de placement dynamique des contrôleurs** adapté aux ressources disponibles
- Proposer un **mécanisme de découverte et de synchronisation** entre les contrôleurs distribués
- Modéliser les **flux de trafic** entre les différentes couches de l'architecture

3. Développer un mécanisme d'équilibrage de charge intelligent basé sur le Machine Learning

- Concevoir un **algorithme de prédiction de charge** basé sur des données historiques de trafic
- Intégrer des techniques de **clustering (ex. K-means)** pour regrouper les flux de données similaires
- Mettre en œuvre un **mécanisme d'allocation dynamique** des requêtes aux contrôleurs selon leur charge
- Adapter les algorithmes d'ordonnancement (Round Robin, aléatoire, pondéré) en fonction des **conditions réseau en temps réel**

4. Optimiser le placement dynamique des contrôleurs SDN

- Développer un modèle de **placement optimal des contrôleurs** minimisant la latence entre commutateurs et contrôleurs
- Intégrer des techniques d'**apprentissage par renforcement** pour adapter automatiquement le placement selon les conditions du réseau
- Gérer dynamiquement le **nombre de contrôleurs actifs** en fonction de la demande en temps réel
- Minimiser le **coût de migration** lors du repositionnement des contrôleurs

5. Améliorer la gestion du trafic réseau par des approches prédictives

- Mettre en place un système de **prévision du trafic** (court terme et long terme) à l'aide de réseaux de neurones ou de modèles LSTM
- Détecter **en temps réel les anomalies** de trafic (attaques DDoS, surcharges soudaines)
- Proposer des mécanismes de **routage adaptatif** basés sur les prédictions du modèle ML
- Assurer une **qualité de service (QoS)** différenciée selon les types de flux

6. Réduire la latence de bout en bout dans l'environnement Cloud-Edge

- Minimiser le **délai de traitement** des requêtes en rapprochant les contrôleurs des nœuds Edge
- Optimiser la **sélection du contrôleur** le plus proche et le moins chargé pour chaque flux
- Réduire les **délais de migration** lors des réassociations commutateur-contrôleur
- Évaluer le **compromis latence / équilibrage de charge** selon différents scénarios

7. Assurer la scalabilité et la résilience de l'architecture

- Concevoir un système capable de **s'adapter automatiquement** à une augmentation du nombre de nœuds Edge
- Mettre en place des mécanismes de **tolérance aux pannes** pour garantir la continuité du service
- Évaluer les performances sous des **charges variables et des conditions réseau dégradées**
- Assurer la **cohérence des données** entre les contrôleurs distribués malgré les mises à jour dynamiques

8. Évaluer et valider les performances du système proposé

- Définir des **métriques de performance** claires : latence, débit, utilisation du CPU, temps de réponse
- Comparer le système proposé avec des approches existantes (ONOS statique, BLAC, etc.)
- Conduire des **simulations** sur des environnements émulés (ex. Mininet) et des scénarios réels
- Analyser les **gains de performance** obtenus grâce aux techniques de ML par rapport aux approches classiques

Conclusion du chapitre

Ce premier chapitre a posé les fondations nécessaires à la compréhension du travail. Il a d'abord présenté le SDN, son architecture en couches et le protocole OpenFlow, puis les paradigmes du Cloud Edge Computing et leurs cas d'usage. L'examen du SDN décentralisé appliqué au Cloud Edge a mis en évidence une tension structurelle : la distribution géographique des contrôleurs améliore la résilience et rapproche le traitement des utilisateurs, mais elle introduit une hétérogénéité des capacités et des latences que les mécanismes d'équilibrage classiques ne prennent pas en compte.

L'analyse des approches d'optimisation a montré les limites des méthodes classiques, à seuils ou heuristiques, face à un trafic non linéaire et imprévisible, et justifié le recours à l'apprentissage automatique. Ces constats ont conduit à la formulation de la problématique et des huit objectifs qui structurent la suite du travail. Le chapitre suivant expose la méthodologie retenue pour y répondre.

CHAPITRE II : METHODOLOGIE

1. État de l'art

L'intégration du Machine Learning (ML) et du Deep Learning (DL) dans le SDN permet de rendre les réseaux intelligents, adaptatifs et proactifs. De nombreuses études récentes démontrent que le ML améliore significativement le routage, la gestion du trafic, l'allocation des ressources, la détection d'anomalies et la sécurité (Faezi & Shirmarz, 2023 ; Muheden et al., 2024 ; Askar et al., 2025).

Plusieurs travaux récents se sont concentrés sur le **routage intelligent**. **Wang et al. (2024)** [31] proposent **DFRDRL** (Dynamic Fuzzy Routing algorithm based on Deep Reinforcement Learning), un algorithme de routage flou dynamique basé sur le Deep Reinforcement Learning (DRL) qui garantit la latence et la bande passante tout en s'adaptant en temps réel aux changements de trafic. L'algorithme réalise un routage en ligne entre paires ingress-egress tout en prédisant en temps réel la matrice de trafic. Il réduit la dépendance à la topologie physique en pondérant le réseau selon l'importance des nœuds critiques. Un mécanisme de déferlement (deferral) permet, en cas de congestion, de prioriser les requêtes à faible consommation de ressources en mettant temporairement en file d'attente les flux les plus exigeants. DFRDRL surpasse les algorithmes existants en termes de latence et de throughput. Il améliore le taux d'admission des requêtes de routage de 2,5 % par rapport au meilleur algorithme de référence, tout en garantissant simultanément des contraintes de latence et de bande passante.

Senthilkumaran et Sankarasubramaninan (2025) [29] introduisent le framework **HCRG** (Hybrid Causal-RL-GNN) combinant Graph Attention Networks pour encoder l'état topologique du réseau de manière contextuelle et un agent Soft Actor-Critic enrichi d'une composante causale permettant de mieux quantifier l'impact des action, ainsi qu'une fonction de récompense composite optimisant simultanément la latence, le taux de perte de paquets et le throughput. Le modèle est entraîné offline sur des topologies NSFNET et Fat-Tree via Mininet, puis déployé en temps réel sur un contrôleur Ryu via OpenFlow. Ce modèle obtient une réduction de latence de 28 %, une augmentation du débit de 22 % et une diminution des pertes

de 35 % par rapport aux baselines (ECMP, ROAR), Une latence d'inférence inférieure à 5 ms, même à grande échelle.

Ali et Awad (2025) [30] ont renforcé le contrôleur POX (largement utilisé en recherche) en y intégrant des métriques de congestion en temps réel, des timeouts adaptatifs et un mécanisme de load balancing. Ils ont ajouté un modèle XGBoost pour classifier en continu l'état du réseau et optimiser les décisions de routage. Ils obtiennent ainsi une latence stable de 3,52 ms, zéro perte de paquets et une précision du modèle de 99,67 % (precision, recall et F1-score à 99 %), avec un modèle très léger (140 KB) adapté au déploiement réel

Hamzah (2025) [25] présente un framework hybride novateur qui intègre simultanément l'apprentissage par renforcement et l'apprentissage non supervisé. Le Deep Q-Network (DQN) est l'algorithme d'apprentissage par renforcement profond utilisé pour l'optimisation dynamique du routage. Il est employé pour résoudre le problème de routage comme un processus de décision séquentielle, permettant au contrôleur SDN de sélectionner dynamiquement les meilleurs chemins en fonction de l'état global du réseau. Il permet au contrôleur SDN d'apprendre de manière autonome les meilleures politiques de routage en fonction de l'état actuel du réseau (charge des liens, latence, bande passante disponible, etc.). Il maximise une fonction de récompense qui prend en compte plusieurs métriques de performance (throughput, latence, taux de perte de paquets). L'**Autoencoder** est un modèle d'apprentissage non supervisé déployé en parallèle pour la **détection d'anomalies** en temps réel. Il est entraîné pour reconstruire le trafic normal et détecter toute déviation considérée comme anomale (anomalies, attaques DDoS, congestions inhabituelles ou comportements malveillants). Cette architecture offre l'avantage de traiter conjointement deux problématiques critiques du SDN : l'optimisation des performances réseau et la détection proactive de menaces. Les expérimentations menées sur des données réelles et synthétiques ont montré une bonne capacité d'adaptation face aux variations de trafic en temps réel, confirmant ainsi la robustesse et la flexibilité du modèle dans des environnements dynamiques. Il permet d'améliorer significativement l'efficacité globale du réseau tout en maintenant une capacité élevée de détection d'anomalies. Cette dualité (optimisation + sécurité) constitue un atout majeur par rapport aux approches qui traitent ces deux aspects séparément.

Dans les réseaux SDN modernes, le trafic est extrêmement hétérogène (flux vidéo, VoIP, IoT, transferts massifs, jeux en ligne, etc.). Chaque type de flux possède des exigences différentes en termes de latence, bande passante, jitter et tolérance aux pertes. Les approches

traditionnelles de routage ne distinguent pas ces flux et appliquent souvent la même politique, ce qui entraîne une mauvaise allocation des ressources et une dégradation de la Quality of Service (QoS) et de la Quality of Experience (QoE) (Amin et al., 2024) [28]. La classification du trafic en temps réel permet au contrôleur SDN d'identifier le type d'application ou le comportement du flux, d'appliquer des politiques de routage dynamiques et adaptées (priorisation, réservation de bande passante, chemins à faible latence, etc.), d'améliorer l'équité entre les flux et l'utilisation globale des ressources réseau. Les travaux récents exploitent principalement des approches d'**apprentissage supervisé** et **profond** pour cette tâche :

- Modèles classiques : Random Forest, Support Vector Machines (SVM) et Decision Trees (Amin et al., 2024) [28].
- Modèles profonds : Convolutional Neural Networks (CNN), Long Short-Term Memory (LSTM) et leurs combinaisons CNN-LSTM pour capturer à la fois les caractéristiques spatiales et temporelles du trafic (Muheden et al., 2024 ; Faezi & Shirmarz, 2023) [32] [26].
- Modèles efficaces pour le temps réel : XGBoost, souvent privilégié pour sa rapidité et sa légèreté, comme dans les travaux d'Ali et Awad (2025) [30] qui l'ont intégré directement dans le contrôleur POX.

Selon Amin et al. (2024), une classification précise du trafic permet d'améliorer significativement les décisions de routage, notamment dans les scénarios multi-contraintes (latence + bande passante). Muheden et al. (2024) insistent quant à eux sur le fait que cette classification constitue un prérequis indispensable à l'implémentation de politiques intelligentes de traffic engineering dans le SDN. La classification du trafic en temps réel apparaît aujourd'hui comme un élément clé de l'intelligence des réseaux SDN, servant de fondation à de nombreuses solutions d'optimisation du routage et de gestion des ressources.

L'un des défis majeurs des SDN multi-contrôleurs est l'**équilibrage de charge** entre les contrôleurs. Les approches traditionnelles de binding statique entraînent souvent une répartition inégale de la charge, tandis que les mécanismes de binding dynamique sont complexes et introduisent une latence supplémentaire **(Aly et al., 2025)** [33]. Ces auteurs proposent une architecture innovante nommée **SDN_CTRLML** (*No binding machine learning architecture for SDN controllers*). Contrairement aux solutions existantes qui reposent sur un binding dynamique coûteux, leur approche introduit une **couche de scheduling** intermédiaire qui utilise des algorithmes de Machine Learning pour assigner dynamiquement les demandes de flux (flow

requests) aux contrôleurs disponibles. Selon les expérimentations, SDN_CTRLML surpasse les modèles de binding statique en termes de répartition de charge, de temps de réponse et d'utilisation des ressources, tout en évitant les inconvénients (latence et complexité) des approches dynamiques classiques. Cette architecture offre un bon compromis entre performance et simplicité de déploiement.

Avec la croissance exponentielle de l'Internet des Objets (IoT) et le développement de l'Edge Computing, l'architecture SDN centralisée classique montre ses limites en termes de latence, bande passante et résilience. Plusieurs travaux mettent en avant l'intérêt d'un **SDN distribué** combiné à des techniques d'intelligence distribuée. **Firouzi et Rahmani (2022)** [27] proposent une **architecture SDN distribuée pour l'IoT** qui permet aux gateways IoT d'effectuer un traitement intelligent à la périphérie du réseau. Au lieu d'envoyer toutes les données vers le cloud, les gateways SDN locales prennent des décisions en fonction de l'état actuel des ressources réseau. Ils intègrent également un algorithme de sélection de clients pour l'entraînement de **Federated Learning (**Apprentissage Fédéré), permettant une collaboration distribuée tout en préservant la confidentialité des données. Les avantages mis en avant sont : une réduction significative de la latence et de la consommation de bande passante, une meilleure résilience face aux pannes et aux partitions réseau et un traitement local des données sensibles tout en bénéficiant de l'intelligence collective via Federated Learning.

Kazi et al. (2025) [34], dans leur survey sur les réseaux Edge Cloud enabled par SDN, confirment cette tendance. Ils soulignent que l'intégration du SDN distribué avec le Federated Learning et l'Edge Computing constitue une solution prometteuse pour répondre aux exigences de faible latence, de scalabilité et de sécurité dans les environnements IoT et IIoT. Selon eux, le SDN distribué facilite l'orchestration, le load balancing et la gestion dynamique des ressources à la périphérie, tandis que le Federated Learning permet d'entraîner des modèles ML sans centraliser les données brutes.

L'approche SDN distribué + Federated Learning apparaît comme particulièrement adaptée aux scénarios Edge/IoT, car elle combine :

- La programmabilité et la visibilité offerte par le SDN,
- L'intelligence distribuée et la préservation de la vie privée via le Federated Learning,
- La réduction des coûts de transmission vers le cloud (Firouzi & Rahmani, 2022 ; Kazi et al., 2025).

Les revues soulignent que le ML est devenu un outil essentiel pour l'évolution du SDN vers des réseaux auto-optimisants et résilients.

Le tableau suivant présente une synthèse comparative des travaux les plus représentatifs issus de la littérature récente sur l'intégration du Machine Learning dans le SDN. Il met particulièrement en évidence les contributions dans les domaines du routage et de la sécurité, tout en couvrant d'autres aspects importants tels que la gestion des contrôleurs et la classification de trafic.

Référence / Travail	Intitulé du travail	Technique principale	Focus principal	Résultats et Limites
Ali et al. (2025)	Enhancing SDN Performance: Machine Learning Integration with the POX Controller	XGBoost + Congestion-Aware Controller	Routage & Gestion de congestion	**Résultats** : Précision globale de 99,67 %, latence moyenne stable à 3,52 ms, **zéro perte de paquets**, throughput amélioré à 9,56 Mbps, modèle très léger (140 KB). **Limites** : Performances moins validées sur de très grandes topologies et environnements réels.
Wang et al. (2024)	DFRDRL: Dynamic Fuzzy Routing Algorithm based on Deep Reinforcement Learning	DRL + Logique Floue	Routage dynamique (garantie latence + bande passante)	**Résultats** : Taux d'admission des requêtes de routage supérieur de 2,5 % par rapport aux meilleurs algorithmes existants, bonne gestion de la congestion via mécanisme de deferral, excellente performance en latence et bande passante. **Limites** : Complexité et temps de convergence élevés lors de l'entraînement DRL.
Senthilkumaran et al. (2025)	ML-Driven Adaptive Routing and	Hybrid Causal-RL-GNN (GAT +	Routage adaptatif avancé &	**Résultats** : Réduction de **28 %** de la latence (22 ms vs 45 ms), augmentation de **22 %** du

	Performance in SDN (HCRG)	SAC)	sensible à la topologie	throughput (2,2 Gbps), réduction de **35 %** de la perte de paquets par rapport aux baselines (ECMP, ROAR, RouteNet), inférence < 5 ms. **Limites** : Overhead computationnel élevé dû à la combinaison GNN + RL.
Hamzah (2025)	Optimizing SDN Performance Through ML-Based Traffic Management	DQN (routage) + Autoencoder (anomalies)	Routage + Détection d'anomalies	**Résultats** : Amélioration significative de l'efficacité du routage, haute précision de détection d'anomalies, bonne adaptabilité aux différents patterns de trafic. **Limites** : Forte dépendance à la qualité et à la diversité des datasets d'entraînement.
Molose & Isong (2026)	A Survey of Multi-Layer IoT Security Using SDN, Blockchain, and ML	SDN + Blockchain + ML/DL (incl. Federated)	**Sécurité multi-couches (IoT/IIoT)**	**Résultats** : Haute précision de détection des menaces, confiance décentralisée, immutabilité des logs et mitigation adaptative grâce à l'approche hybride SDN-BC-ML. **Limites** : Overhead important induit par la Blockchain et défis de scalabilité sur déploiements à grande échelle.
Aly et al. (2025)	No binding machine learning architecture for SDN	ML Scheduling (No-binding)	Load balancing des contrôleurs	**Résultats** : Meilleure répartition des requêtes de flux entre contrôleurs sans mécanisme de migration dynamique, réduction notable

	controllers			de la latence et de la complexité opérationnelle. **Limites** : Approche principalement centrée sur la gestion des contrôleurs, moins orientée sur le routage des flux de données.
Application-Aware Routing	Machine Learning–Assisted Application-Aware Routing Using Traffic Classification	Classification ML (SVM, RF, CNN) + SDN	**Routage Application-Aware**	**Résultats** : Amélioration significative de la QoS (latence, bande passante, perte de paquets et équité) selon le type d'application, meilleure utilisation des ressources réseau. **Limites** : Les performances globales dépendent fortement de la précision de la classification de trafic.

Tableau 1 : Tableau comparatif des approches ML dans le SDN

2. Notre approche : équilibrage de charge

2.1. Analyse de l'existant : approche SDN_CTRLML

Parmi les travaux récents portant sur l'équilibrage de charge des contrôleurs dans les architectures SDN distribuées, l'article de **Aly et al. (2025)**, intitulé *« No binding machine learning architecture for SDN controllers »*, propose une contribution notable. Les auteurs introduisent **SDN_CTRLML (**SDN Controller Machine Learning), une architecture sans liaison (*bindingless*) qui intègre une couche de planification intelligente basée sur l'apprentissage automatique, plus précisément l'algorithme de **K-means**.

Cet article introduit SDN_CTRLML, une nouvelle architecture pour contrôleurs distribués qui utilise des approches d'apprentissage automatique IA pour l'équilibrage de charge. SDN_CTRLML assure l'équilibrage de charge entre contrôleurs grâce à une couche de planification qui permet l'association commutateur-contrôleur sans liaisons fixes. Cela permet aux commutateurs d'envoyer des messages à différents contrôleurs en fonction des algorithmes de planification sans interrompre la ré-association des commutateurs. Cela permet de maintenir les performances du réseau et de minimiser les délais globaux.

Cette architecture vise à résoudre le problème de déséquilibre de charge inhérent aux approches traditionnelles à liaison statique, tout en évitant la complexité et la latence induites par les mécanismes de migration dynamique de commutateurs. SDN_CTRLML agit comme une couche intermédiaire transparente qui intercepte les paquets entrant des commutateurs et les répartit dynamiquement vers les contrôleurs en utilisant des algorithmes de planification (Round-Robin, Randomized, Weighted Round-Robin, etc.), optimisés par le regroupement K-means sur les données historiques de charge. Les résultats expérimentaux rapportés montrent une amélioration significative par rapport au modèle de référence BLAC et à ONOS statique, avec une réduction du temps de réponse de l'ordre de 13 % et une augmentation de l'utilisation des ressources de 14 %.

Les infrastructures Cloud-Edge décentralisées présentent des caractéristiques distinctes qui complexifient considérablement le problème d'équilibrage de charge des contrôleurs SDN par rapport aux environnements de data centers traditionnels. Bien que cette approche soit novatrice, elle présente certaines limites dans le contexte des infrastructures Cloud-Edge décentralisées visé par notre travail.

- **L'hétérogénéité des ressources constitue un défi majeur**

Le SDN_CTRLML repose principalement sur une métrique unique (l'utilisation CPU) et suppose une homogénéité des contrôleurs, hypothèse qui ne tient plus dans un environnement Cloud-Edge où coexistent des nœuds Edge aux ressources contraintes et des nœuds Cloud plus puissants, avec des disparités importantes en termes de latence, bande passante et consommation énergétique. Les nœuds Edge disposent de capacités computationnelles, mémoire et énergétiques limitées, tandis que les nœuds Cloud offrent une puissance importante

mais à une latence plus élevée. Cette disparité rend les approches basées sur une métrique unique (comme l'utilisation CPU dans SDN_CTRLML) inadaptées.

L'approche utilise un K-means classique sans mécanisme de prédiction de charge future ni prise en compte de la localisation géographique et de la latence réelle entre commutateurs et contrôleurs. Or, dans les environnements distribués Cloud-Edge, la distance physique et les variations de latence jouent un rôle critique.

Enfin, la gestion des flux dépendants et la dynamique élevée propre aux environnements Edge (mobilité, variation brutale de charge) ne sont que partiellement abordées.

Ces limites justifient la nécessité d'adapter et d'améliorer l'architecture SDN_CTRLML afin de la rendre adaptée aux exigences spécifiques des infrastructures Cloud-Edge décentralisées, notamment en termes d'hétérogénéité, de multi-objectifs d'optimisation et de scalabilité géographique.

Deuxièmement, la **distribution géographique** et la **dynamique élevée** du trafic caractérisent ces environnements. Les applications IoT, véhicules autonomes, réalité augmentée et analyse vidéo en temps réel génèrent des variations brutales et localisées de charge, ainsi qu'une mobilité fréquente des utilisateurs et des dispositifs. Ces phénomènes entraînent une obsolescence rapide des décisions d'affectation statiques ou semi-statiques.

Troisièmement, les **contraintes multi-objectifs** deviennent prépondérantes : il ne s'agit plus seulement de maximiser l'utilisation des contrôleurs, mais de trouver un compromis optimal entre plusieurs critères contradictoires tels que la latence de réponse, la consommation énergétique des nœuds Edge, la bande passante inter-domaines, la fiabilité et la résilience face aux défaillances.

Enfin, la **gestion de la consistance** et des **flux dépendants** dans un contexte hautement distribué pose des difficultés supplémentaires. Les mécanismes de réplication de l'état réseau (utilisés par ONOS ou ONIX) peuvent générer une surcharge significative lorsque les contrôleurs sont géographiquement éloignés.

Ces problématiques révèlent les limites des architectures existantes, y compris SDN_CTRLML, et justifient le développement d'une solution spécifiquement adaptée aux environnements Cloud-Edge décentralisés.

2.2. Le SDN_CTRLML_Edge

Dans le cadre de notre travail sur l'**optimisation du SDN décentralisé dans une infrastructure Cloud-Edge**, nous proposons d'étendre l'approche de Aly et al. (2025) en concevant **SDN_CTRLML-Edge**, une version adaptée aux contraintes spécifiques des environnements distribués.

Le SDN_CTRLML-Edge est une architecture SDN décentralisée basée sur EMA et K-means++ pour les environnements Cloud-Edge distribués. Contrairement aux approches traditionnelles basées sur une association statique entre les équipements réseau et les contrôleurs SDN, SDN_CTRLML-Edge conserve le principe de l'architecture sans liaison (Bindingless Architecture) et introduit un mécanisme intelligent de prise de décision reposant sur l'algorithme K-means++ et la moyenne mobile exponentielle (EMA).

L'objectif principal de cette architecture est d'améliorer l'utilisation des ressources Edge, de réduire la surcharge des nœuds, de minimiser les temps de réponse des services et d'assurer une meilleure répartition de la charge dans un environnement distribué.

2.3. Architecture globale

L'architecture proposée est composée de quatre couches principales :

a. Couche Infrastructure Cloud-Edge

Cette couche représente les ressources physiques et virtuelles du système :

- Centres de données Cloud ;
- Serveurs Fog ;
- Nœuds Edge ;
- Machines virtuelles ;
- Conteneurs ;
- Commutateurs OpenFlow.

Les ressources sont géographiquement distribuées et hébergent les applications utilisateurs.

b. Couche Contrôle SDN Décentralisé

Chaque domaine Edge est administré par un contrôleur SDN local qui assure :

- la collecte des statistiques réseau ;
- la gestion des flux OpenFlow ;
- l'allocation des ressources ;
- l'équilibrage de charge local.

Contrairement aux architectures SDN distribuées classiques, aucun contrôleur central n'est responsable de l'ensemble du réseau. Chaque contrôleur prend ses décisions de manière autonome tout en échangeant périodiquement des informations résumées avec les autres domaines.

c. Couche d'Analyse Intelligente

Cette couche constitue la contribution principale de SDN_CTRLML-Edge.

Elle est composée de deux modules :

❖ Module EMA

Le module EMA réalise le lissage temporel des métriques collectées.

Son rôle est de :

- réduire l'influence des fluctuations momentanées ;
- identifier les tendances réelles de charge ;
- fournir des données plus stables au moteur de classification.

Les métriques surveillées sont :

- utilisation CPU ;
- utilisation mémoire ;
- bande passante ;
- latence ;
- nombre de flux ;
- taux de paquets Packet-In.

❖ Module K-means++

Le module K-means++ réalise une classification dynamique des ressources Edge. Chaque nœud est représenté par un vecteur contenant les valeurs EMA des différentes métriques. Le mécanisme de clustering permet de regrouper automatiquement les ressources en plusieurs catégories :

- ressources sous-utilisées ;
- ressources normalement chargées ;
- ressources surchargées.

Cette classification est continuellement mise à jour afin de refléter l'évolution du trafic réseau.

d. Couche Décisionnelle

Cette couche exploite les résultats du clustering afin de prendre des décisions d'optimisation. Les principales décisions concernent :

- ✓ le placement des services ;
- ✓ l'affectation des flux ;
- ✓ la migration des charges de travail ;
- ✓ l'équilibrage des ressources ;
- ✓ l'optimisation des chemins de communication.

Les ressources appartenant aux clusters les moins chargés sont privilégiées pour accueillir les nouveaux services.

2.4. Fonctionnement de la solution SDN_CTRLML-Edge

Le fonctionnement de SDN_CTRLML-Edge suit un pipeline en boucle fermée, exécuté périodiquement par chaque contrôleur SDN. Les métriques collectées sur les équipements du domaine sont lissées par EMA, classées par K-means++ en trois profils de charge, puis exploitées par la couche décisionnelle pour rééquilibrer les flux. La figure 8 en résume les cinq étapes.

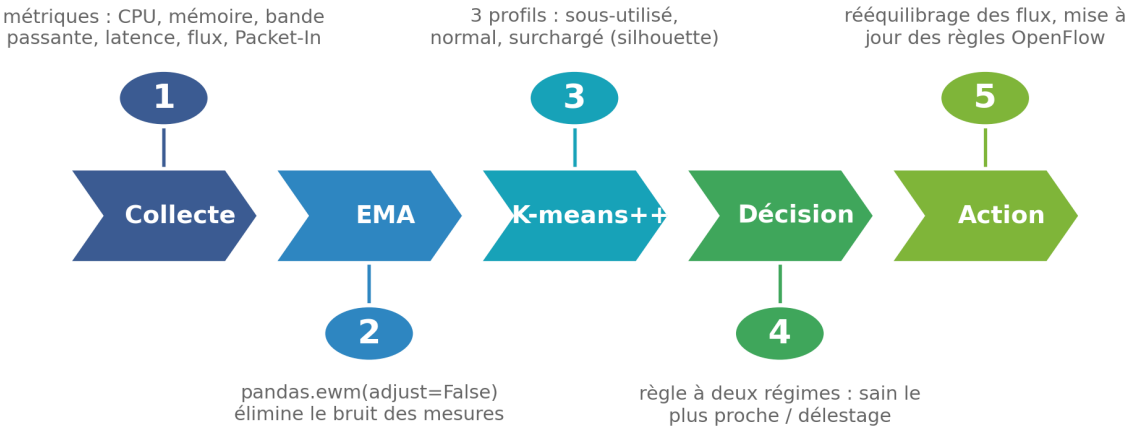


***Figure 8** : Pipeline de fonctionnement de la solution SDN_CTRLML-Edge*

Contrairement à SDN_CTRLML, qui se limite au choix d'un contrôleur, la couche décisionnelle de SDN_CTRLML-Edge prend une décision adaptée à la situation observée dans le domaine Edge. Le tableau 2 récapitule les principales situations et les décisions associées.

Situation	**Décision**
Nouveau flux	Choisir le meilleur nœud Edge du domaine
Nouveau service	Choisir le serveur Edge le plus adapté
Nœud surchargé	Déclencher une migration locale
Bande passante insuffisante	Sélectionner un autre chemin ou un autre nœud
Déséquilibre de charge	Redistribuer les ressources dans le domaine

***Tableau 2** : Décisions de la couche décisionnelle selon la situation observée*

<u>Étape 1</u> : Collecte des métriques

Chaque contrôleur SDN collecte périodiquement les informations provenant des équipements de son domaine :

- charge CPU ;
- occupation mémoire ;
- débit réseau ;
- latence ;

- nombre de flux actifs.

<u>Étape 2</u> : Prétraitement par EMA

Les métriques brutes collectées (CPU, latence, débit...) sont instables par nature, un pic de CPU à t=5s peut être un artefact momentané, pas une tendance réelle. Envoyer ces valeurs brutes à K-means++ produirait des clusters instables et incohérents. Ces valeurs collectées sont traitées à l'aide de la moyenne mobile exponentielle (EMA)

Cette étape permet d'obtenir des indicateurs représentatifs de l'état réel du système tout en éliminant les variations transitoires.

Formule :

$$\mathbf{EMA_t = \alpha X_t + (1-\alpha) EMA_{t-1}}$$

- EMA_t est la valeur de l'EMA au temps actuel (ce qu'on veut calculer)
- α est le facteur de lissage (entre 0 et 1). Le paramètre α contrôle l'équilibre entre réactivité et stabilité :
  - α proche de 1 : très réactif, suit de près X_t, peu de lissage ;
  - α proche de 0 : très lisse, mémoire longue, réagit lentement ;
  - α = 0.1 à 0.3: bon compromis pour métriques réseau.
- X_t est la nouvelle mesure (ex: charge CPU actuelle du contrôleur)
- EMA_{t-1} est la valeur de l'EMA au temps précédent
- $(1-\alpha)$ est le poids donné à l'ancienne valeur

Vecteur de sortie de l'EMA

Pour chaque nœud i, on obtient un vecteur lissé :

$$\mathbf{v_i = [EMA_i^{CPU}, EMA_i^{Mem}, EMA_i^{Débit}, EMA_i^{Latence}, EMA_i^{Flux}]}$$

C'est ce vecteur, stable et représentatif qui est transmis à K-means++.

<u>Étape 3</u> : Classification par K-means++

K-means++ reçoit l'ensemble des vecteurs EMA de tous les nœuds du domaine :

$X=\{v1,v2,\ldots,vn\}$

<u>Phase 1</u> : **Initialisation intelligente** (le "+" de K-means++)

- Centroïde c_1 : choisi aléatoirement parmi les nœuds
- Centroïde c_2 : choisi avec probabilité $\propto D(v_i)^2$ (distance au c_1)
- Centroïde c_k : toujours le plus éloigné des centroïdes déjà placés

Résultat : les centroïdes initiaux couvrent bien l'espace des profils de charge.

Phase 2 : Itérations jusqu'à convergence

RÉPÉTER :

① Assignation : chaque nœud rejoint le cluster dont le centroïde est le plus proche (distance euclidienne sur les EMA)

② Mise à jour : recalculer chaque centroïde = moyenne des membres

JUSQU'À : stabilité des centroïdes

Résultat : les clusters obtenus

Étape 4 : Analyse de l'état du domaine Edge

Dans SDN_CTRLML-Edge, les clusters représentent naturellement des niveaux de charge

Le tableau 3 précise l'interprétation retenue pour chaque cluster de charge.

Cluster	Profil centroïde	Interprétation
C_1	CPU faible, latence faible, peu de flux	Nœuds sous-utilisés
C_2	CPU modérée, débit stable	Nœuds en charge normale
C_3	CPU élevée, latence haute, beaucoup de flux	Nœuds surchargés

***Tableau 3** : Interprétation des clusters de charge*

Le nombre K de clusters est un paramètre de conception ; typiquement K=3 pour capturer les niveaux : sous-chargé, normal et surchargé

Étape 5 : Prise de décision

Mécanisme de décision : après la classification, le contrôleur SDN dispose d'une carte dynamique de l'état du domaine. Il raisonne en trois étapes :

Étape A : Identifier les nœuds surchargés

Un nœud i est déclaré surchargé si son vecteur EMA appartient au cluster de charge élevée ET si ses métriques dépassent des seuils définis :

$$\text{Surchargé}(i) \iff \mathbf{v}_i \in C_{\text{élevé}} \quad \text{ET} \quad \text{EMA}_i^{\text{CPU}} > \theta_{\text{CPU}}$$

Étape B : Identifier les nœuds sous-utilisés (candidats)

Un nœud j est disponible pour absorber du trafic si :

$$\text{Disponible}(j) \iff \mathbf{v}_j \in C_{\text{faible}} \quad \text{ET} \quad \text{EMA}_j^{\text{CPU}} < \theta_{\text{min}}$$

Étape C : Choisir le meilleur nœud cible

Parmi tous les nœuds disponibles, le contrôleur sélectionne celui dont le centroïde est le plus proche du profil requis par le flux à rediriger :

$$j^* = \arg\min_{j \in C_{\text{faible}}} \|\mathbf{v}_j - \mathbf{c}_{\text{faible}}\|^2$$

C'est le nœud le plus représentatif de son cluster, le plus "central", donc le plus stable et prévisible.

La figure 9 schématise cette prise de décision.

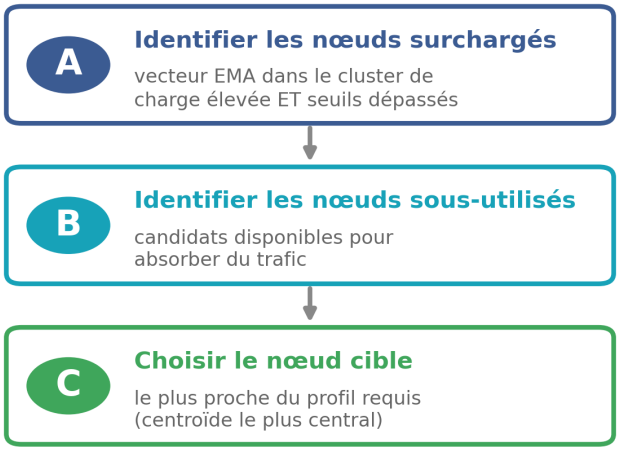


***Figure 9** : Schéma de la prise de décision (Étape 5)*

Étape 6 : Réévaluation continue

Le processus est exécuté périodiquement afin de s'adapter aux variations du trafic et aux changements de charge.

2.5. Implémentation concrète de la couche décisionnelle : règle à deux régimes

La règle de sélection présentée à l'Étape C (nœud le plus proche du profil requis) est volontairement formulée de façon générale pour illustrer le principe. Son implémentation logicielle (module de décision du simulateur) la précise sous la forme d'une règle à deux régimes, retenue après une phase de vérification de cohérence menée à plusieurs taux d'arrivée (4 000, 16 000 et 32 000 requêtes/s) : une préférence systématique pour le contrôleur le plus proche, même parmi des contrôleurs seulement partiellement chargés, conduisait à saturer les petits contrôleurs Edge avant de solliciter les nœuds Fog/Cloud, un comportement contraire à l'objectif d'équilibrage.

Régime normal : parmi les contrôleurs qui ne figurent pas dans le cluster K-means++ jugé surchargé et dont la charge CPU individuelle reste sous un seuil absolu (0,70), le contrôleur retenu est le plus proche en latence réseau (arrondie à la milliseconde ; en deçà, la mesure n'est que du bruit) ; les ex æquo sont départagés par la charge la plus faible.

Régime saturé : si aucun contrôleur ne satisfait ce critère de santé (par exemple lors d'un pic de charge généralisé), le mécanisme bascule vers un délestage pur : le flux est envoyé au contrôleur de charge la plus faible, sans tenir compte de la latence. Ce second régime traduit l'idée que, lorsque tous les contrôleurs approchent la saturation, le temps d'attente en file domine largement le temps de propagation réseau ; s'obstiner sur la proximité reviendrait alors à concentrer davantage la charge sur un nœud déjà critique.

Formellement, pour un flux dont le commutateur source est src, et Ω l'ensemble des contrôleurs candidats (hors cluster surchargé) :

Le logigramme de la figure 10 détaille la règle à deux régimes telle qu'elle est implémentée.

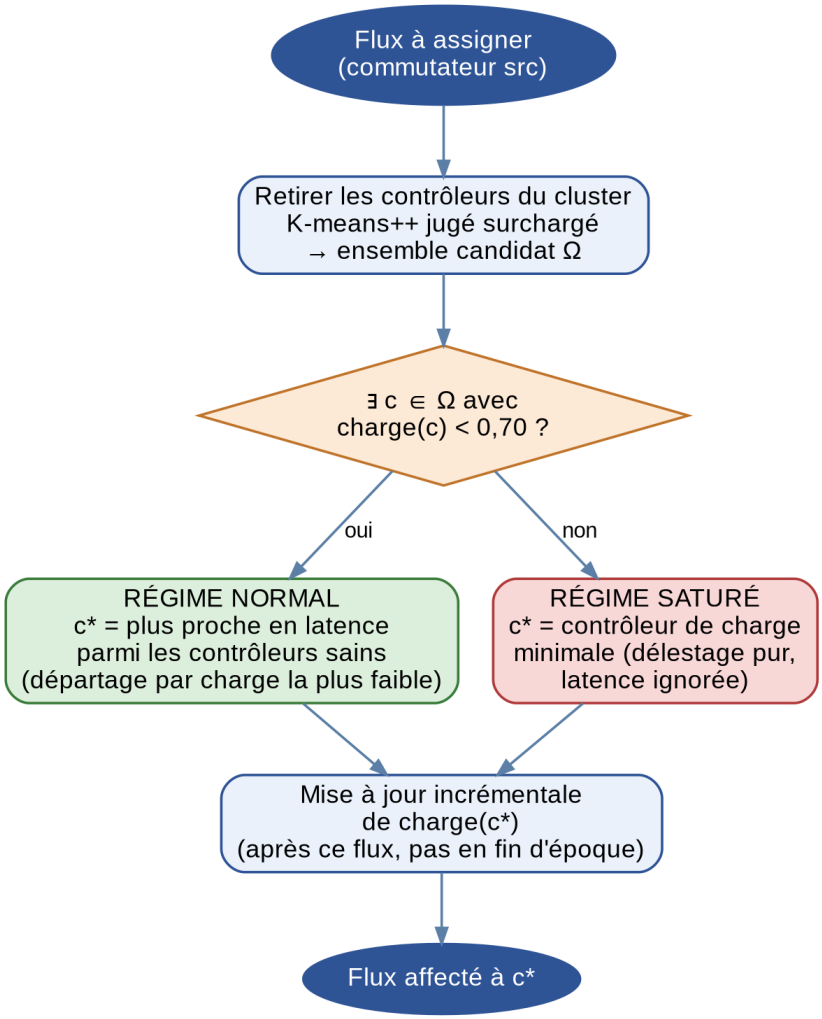


***Figure 10** : Logigramme de la règle de décision à deux régimes implémentée dans le simulateur*

La fonction charge(c) est mise à jour de façon incrémentale après chaque flux, et non uniquement en fin d'époque (200 flux), afin d'éviter un effet de troupeau où l'ensemble d'une époque serait dirigé vers le même contrôleur avant que son estimation de charge ne soit rafraîchie ; un piège identifié lors de la comparaison avec la baseline SDN_CTRLML, dont l'implémentation d'origine ne réévalue la charge qu'à chaque ré-exécution du K-means++.

2.6. Métriques d'évaluation

L'évaluation de SDN_CTRLML-Edge repose sur trois familles de métriques, détaillées ci-dessous : une métrique de qualité du clustering, des métriques opérationnelles mesurées lors de la simulation, et un protocole statistique garantissant la significativité des comparaisons entre stratégies.

2.6.1. Métrique d'évaluation du clustering (K-means++)

Le choix de K = 3 (sous-utilisé / normal / surchargé) n'est pas arbitraire : il est validé empiriquement par le score de silhouette (fonction silhouette_score de scikit-learn), qui mesure pour chaque point la cohésion à son propre cluster relativement à la séparation avec le cluster le plus proche (valeur comprise entre -1 et 1 ; un score proche de 1 indique des clusters bien séparés). Ce score est recalculé à chaque ré-exécution du K-means++ (toutes les 200 affectations de flux) et journalisé ; il permet de vérifier que les trois profils de charge restent statistiquement distincts tout au long de la simulation, et pas seulement au moment de leur initialisation.

2.6.2. Métriques opérationnelles

Pour chaque simulation, les métriques suivantes sont mesurées et moyennées sur l'ensemble des flux (ou des contrôleurs) :

- **Temps de réponse moyen (ms) :** somme de la latence réseau (plus court chemin sur la topologie, calculé avec NetworkX) et du délai d'attente en file M/M/1, $(1000/\mu c)/(1-\rho c)$, du contrôleur retenu ;
- **Débit (flux servis/s) :** proportion de flux traités sans que le contrôleur cible n'atteigne la saturation ($\rho \geq 0{,}999$) ;
- **Utilisation CPU moyenne :** charge moyenne des contrôleurs en fin de simulation ;
- **Indice d'équité de Jain :** mesure la répartition de la charge entre contrôleurs : $J(x) = (\Sigma xi)^2 / (n \cdot \Sigma xi^2)$, bornée entre 1/n (charge concentrée sur un seul contrôleur) et 1 (charge parfaitement égale entre les n contrôleurs) ;
- **Coût de migration :** nombre de réaffectations d'un commutateur source vers un contrôleur différent de celui utilisé lors de sa précédente affectation.

2.6.3. Protocole statistique

Un run isolé ne permet pas de conclure : la charge de fond, la topologie et le trafic comportent une part d'aléa. Pour rendre les résultats défendables, chaque combinaison (stratégie, taux d'arrivée) est répétée sur 25 graines aléatoires (seeds) indépendantes, qui font varier conjointement la topologie, la charge de fond et le trafic, et pas seulement le bruit d'un même scénario. Deux traitements statistiques sont appliqués (bibliothèque SciPy, sous-module stats) :

- un intervalle de confiance à 95 % (loi de Student) est calculé pour chaque métrique, par stratégie et par taux d'arrivée ;
- un test de Wilcoxon apparié compare, graine par graine, SDN_CTRLML-Edge à chacune des quatre stratégies de référence, pour chaque métrique ; le taux de victoire (win rate) et la p-value sont rapportés, avec un seuil de significativité fixé à 5 %.

Données réutilisées et plan d'expériences

Conformément au principe de réutilisation retenu pour ce travail (s'appuyer sur des données et des outils existants plutôt que de tout générer), trois jeux de données réels alimentent, ou peuvent alimenter, le simulateur :

- **Alibaba Cluster Trace** (trace server_usage, github.com/alibaba/clusterdata) : séries temporelles réelles d'utilisation CPU et mémoire sur un parc de machines hétérogènes, utilisées pour calibrer la charge de fond des contrôleurs ;
- **InSDN** : jeu de données SDN au niveau des flux (débit, nombre de flux, taux de Packet-In), utilisé pour calibrer la distribution du trafic injecté ;
- **Internet Topology Zoo** (topology-zoo.org) : plus de 250 topologies réelles d'opérateurs au format GraphML avec coordonnées géographiques, utilisées pour construire la topologie réseau et les latences commutateur-contrôleur (plus court chemin, NetworkX).

Chaque chargeur de données accepte le fichier réel s'il est déposé dans le dossier de données du projet ; en son absence, il retombe sur un générateur synthétique calibré sur les ordres de grandeur publiés de ces jeux de données, ce qui garantit un pipeline exécutable hors ligne. À la date de rédaction, les 1 000 simulations rapportées au chapitre suivant ont été produites avec ce repli synthétique : le raccordement aux fichiers réels est une étape de robustesse supplémentaire, pas un prérequis au fonctionnement du pipeline, mais constitue une limite explicitement assumée des résultats actuels.

Plan d'expériences (Design of Experiments)

La figure 11 récapitule le pipeline expérimental complet, des données au protocole statistique.

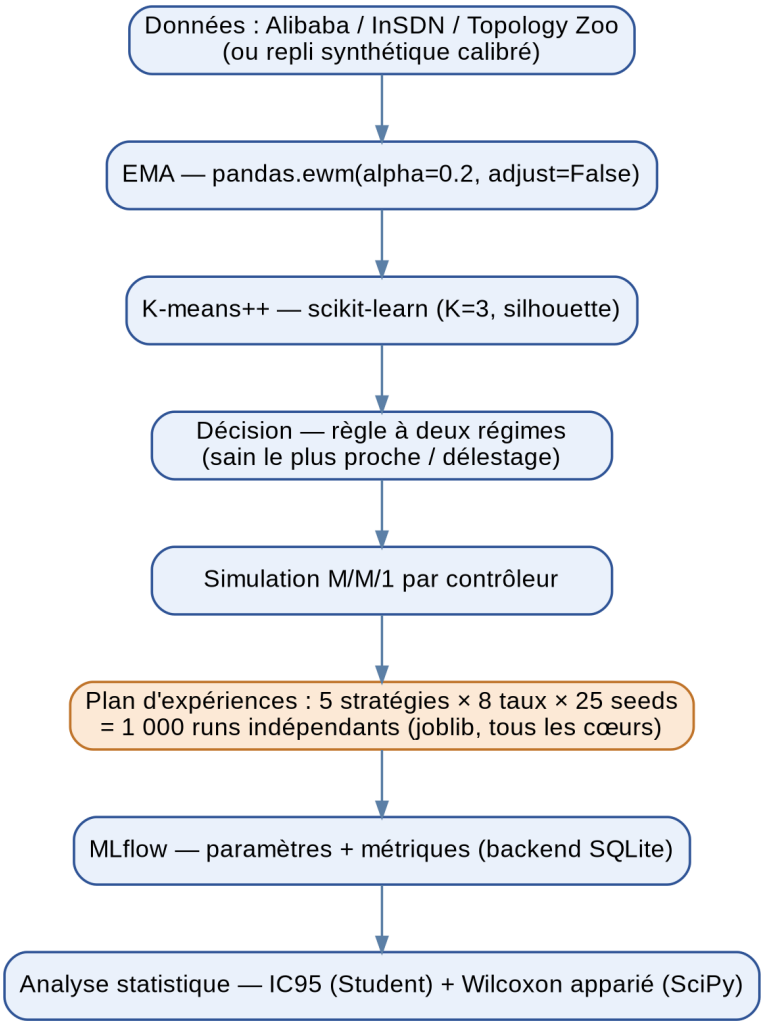


***Figure 11** : Pipeline expérimental complet, des données au protocole statistique*

L'évaluation croise 5 stratégies (SDN_CTRLML-Edge, ONOS statique, SDN_CTRLML, Round-Robin, Random) × 8 taux d'arrivée (de 2 000 à 32 000 requêtes/s) × 25 graines aléatoires, soit 1 000 simulations indépendantes de 6 000 flux chacune. Ces exécutions étant indépendantes, elles sont parallélisées avec la bibliothèque joblib sur l'ensemble des cœurs de la machine (8 cœurs dans notre environnement), chaque worker étant limité à un seul thread BLAS/OpenMP pour éviter la sur-souscription.

Ce choix a été préféré à un orchestrateur de type Airflow ou à un moteur de calcul distribué comme Spark : la charge de travail (des simulations Python indépendantes sur une seule machine) se parallélise trivialement sur les cœurs disponibles, sans qu'un cluster ou un ordonnanceur dédié n'apporte de bénéfice proportionné à sa complexité opérationnelle. La rigueur des conclusions vient du multi-seed et du protocole statistique, pas d'une infrastructure de calcul distribué.

Chaque exécution est journalisée dans MLflow (paramètres et métriques), avec un backend de suivi SQLite, le magasin de fichiers utilisé par défaut par MLflow ayant été déprécié à partir de la version 3. Cela permet d'explorer les 1 000 runs via l'interface MLflow sans dépendance à un serveur de tracking externe.

Outils et bibliothèques utilisées

Environnement matériel et logiciel

Le tableau 4 décrit l'environnement matériel et logiciel des expériences.

Élément	Caractéristique
Processeur	Intel Core i5-8265U @ 1,60 GHz (8 threads)
Mémoire vive	23 Go
Système d'exploitation	Ubuntu 24.04 LTS
Langage	Python 3.12, environnement virtuel venv
Structure de projet	Cookiecutter Data Science

***Tableau 4** : Environnement matériel et logiciel*

Bibliothèques Python

Les bibliothèques employées sont listées dans le tableau 5.

Bibliothèque	Version	Rôle dans le projet
pandas	2.3	Lissage EMA (ewm avec adjust=False), manipulation des séries de métriques
NumPy	1.26	Calcul vectoriel
scikit-learn	1.9	K-means++ (KMeans), normalisation (StandardScaler), score de silhouette
NetworkX	3.6	Topologie réseau (GraphML), plus courts chemins (latences)
SciPy	1.11	Statistiques : IC95 (loi de Student), test de Wilcoxon apparié
joblib	1.5	Parallélisation des 1 000 simulations sur les cœurs

		disponibles
MLflow	3.14	Suivi des expériences (paramètres, métriques), backend SQLite
Matplotlib / Seaborn	3.6 / 0.13	Génération des figures (bandes IC95, boîtes à moustaches)
pytest	9.1	Tests de vérification bout-en-bout (6 tests)

***Tableau 5** : Bibliothèques Python utilisées*

Aucune de ces briques n'a été redéveloppée : chacune correspond à une bibliothèque de référence de l'écosystème scientifique Python, conformément au principe de réutilisation retenu pour ce travail. Le seul moteur ad hoc du projet est la simulation elle-même : une file M/M/1 par contrôleur, plus simple et suffisante pour ce périmètre qu'un moteur à événements discrets complet (tel que SimPy), qui resterait pertinent si le protocole expérimental venait à intégrer des dépendances entre flux ou des files d'attente plus réalistes.

Conclusion du chapitre

Ce chapitre a exposé la démarche suivie pour concevoir et évaluer SDN_CTRLML-Edge. L'état de l'art a situé la contribution parmi les travaux récents d'intégration de l'apprentissage automatique au SDN, et identifié dans SDN_CTRLML (Aly et al., 2025) le prédécesseur direct ainsi que ses deux limites en contexte Cloud-Edge : une métrique unique et l'hypothèse de contrôleurs homogènes.

L'architecture proposée a ensuite été décrite couche par couche, depuis l'infrastructure hétérogène jusqu'à la couche décisionnelle, en détaillant le lissage EMA, la classification K-means++ et la règle de décision à deux régimes effectivement implémentée. Le protocole d'évaluation a enfin été défini : jeux de données réutilisés, plan d'expériences de 1 000 simulations, métriques retenues et traitement statistique par intervalles de confiance et tests de Wilcoxon appariés. Le chapitre suivant présente et commente les résultats obtenus.

CHAPITRE III : RESULTATS ET COMMENTAIRES

Ce chapitre présente et commente les résultats obtenus par le simulateur décrit au chapitre précédent. L'objectif est de vérifier, sur des bases statistiques solides, si SDN_CTRLML-Edge tient ses promesses face à son prédécesseur direct SDN_CTRLML et face aux stratégies de référence. Les résultats sont d'abord présentés globalement, puis métrique par métrique, avant d'être validés par des tests de significativité et discutés de façon critique.

3.1. Rappel des conditions expérimentales

L'ensemble des résultats présentés dans ce chapitre provient d'un plan d'expériences unique, exécuté en une seule campagne. Le tableau 6 en rappelle les paramètres.

Paramètre	Valeur
Stratégies comparées	5 (proposée + 4 références)
Taux d'arrivée	8 points, de 2 000 à 32 000 requêtes/s
Répétitions (graines aléatoires)	25 par configuration
Nombre total de simulations	1 000
Flux simulés par exécution	6 000
Contrôleurs	10 (répartis cloud / fog / edge)
Coefficient de lissage EMA	$\alpha = 0,2$
Nombre de clusters K-means++	$K = 3$
Seuil absolu de surcharge	0,70
Intervalles de confiance	95 % (loi de Student) sur les 25 graines
Test de significativité	Wilcoxon apparié, seuil 5 %

***Tableau 6** : Paramètres du plan d'expériences*

Chaque valeur rapportée dans les figures et tableaux qui suivent est donc une moyenne sur 25 exécutions indépendantes, accompagnée de son intervalle de confiance à 95 %. Les bandes colorées autour des courbes représentent ces intervalles : lorsqu'elles ne se recouvrent pas, l'écart observé entre deux stratégies n'est pas imputable au hasard des tirages.

3.2. Vue d'ensemble des performances

Le tableau 7 présente les moyennes globales de chaque stratégie, toutes charges confondues. Il donne une première lecture d'ensemble avant l'analyse détaillée.

Stratégie	Temps de réponse (ms)	Débit (flux/s)	Utilisation CPU	Indice de Jain	Réaffec-tations
SDN_CTRLML-Edge (proposée)	**77,8**	**8 022**	**0,594**	**0,978**	**2 478**
SDN_CTRLML (CPU seul)	87,8	7 643	0,607	0,979	3 864
ONOS statique	182,2	4 377	0,718	0,949	0
Round-Robin	242,1	4 485	0,740	0,950	5 426
Random	239,3	4 514	0,735	0,952	5 361

***Tableau 7** : Moyennes globales par stratégie sur les 1 000 exécutions*

La figure 12 compare les moyennes globales de temps de réponse et de débit.

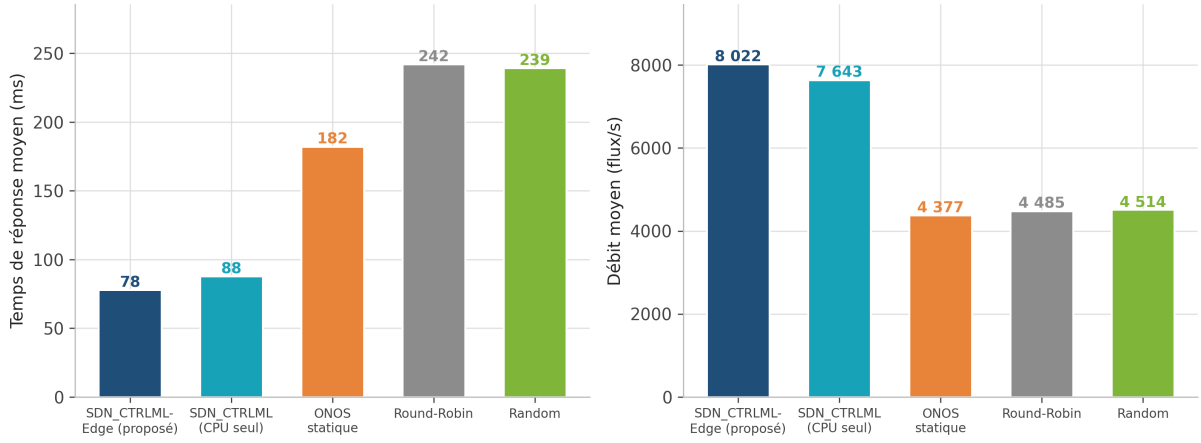


***Figure 12** : Comparaison des moyennes globales de temps de réponse et de débit*

La stratégie proposée obtient le meilleur temps de réponse moyen (77,8 ms) et le meilleur débit moyen (8 022 flux/s). Rapportés aux références, ces résultats correspondent à une réduction du temps de réponse de **57,3 % face à ONOS statique**, de **67,9 % face à Round-Robin** et de **11,3 % face à SDN_CTRLML**, le prédécesseur direct. Le débit progresse quant à lui de 83,3 % face à ONOS et de 5,0 % face à SDN_CTRLML.

Ce dernier écart mérite d'être souligné : l'article de référence (Aly et al., 2025) revendique une amélioration d'environ 13 % du temps de réponse de SDN_CTRLML par rapport à ses

propres références. Le gain supplémentaire de 11,3 % obtenu ici par SDN_CTRLML-Edge sur SDN_CTRLML est donc du même ordre de grandeur, ce qui situe l'apport de l'extension multi-métriques au niveau de celui de l'architecture d'origine sur l'état de l'art antérieur.

3.3. Temps de réponse

La figure 13 détaille l'évolution du temps de réponse en fonction de la charge injectée. C'est la métrique la plus révélatrice des différences entre stratégies.

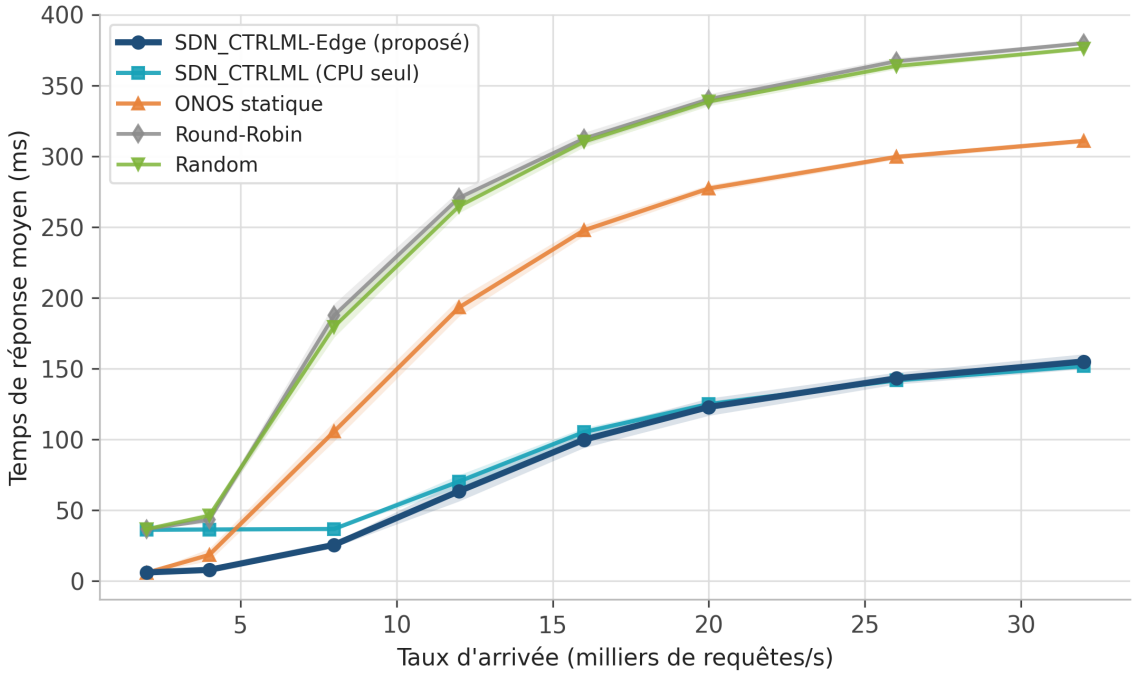


***Figure 13** : Temps de réponse moyen en fonction du taux d'arrivée (moyenne et intervalle de confiance à 95 %)*

Trois régimes se distinguent nettement.

- **En charge faible (2 000 à 4 000 requêtes/s) :** toutes les stratégies conscientes de la latence restent sous 20 ms. SDN_CTRLML, qui ignore la localisation géographique, plafonne en revanche à 36 ms car il répartit les flux sur le contrôleur le moins chargé sans tenir compte de la distance. À 2 000 requêtes/s, ONOS statique est même très légèrement meilleur que la stratégie proposée (5,6 ms contre 5,9 ms) : lorsque aucun contrôleur n'est saturé, une liaison fixe vers le contrôleur le plus proche est optimale et le mécanisme de rééquilibrage n'apporte rien.
- **En charge intermédiaire (8 000 à 20 000 requêtes/s) :** c'est là que l'écart se creuse le plus. ONOS statique se dégrade très vite (de 105 à 277 ms) car ses

liaisons figées concentrent la charge sur les contrôleurs les plus sollicités, sans possibilité de report. La stratégie proposée passe de 25 à 123 ms seulement, soit un temps de réponse divisé par plus de deux face à ONOS sur toute cette plage.

- **En saturation (26 000 à 32 000 requêtes/s) :** les courbes de SDN_CTRLML-Edge et de SDN_CTRLML convergent, et SDN_CTRLML devient même marginalement meilleur (151,6 ms contre 154,9 ms à 32 000 requêtes/s). Ce résultat s'explique : lorsque tous les contrôleurs approchent la saturation, la stratégie proposée bascule dans son régime de délestage et privilégie l'équilibrage à la proximité, ce qui coûte quelques millisecondes de latence réseau. Ce léger retrait est le prix assumé d'un bien meilleur équilibrage, comme le montre la section suivante.

Les stratégies naïves Round-Robin et Random restent les plus mauvaises sur toute la plage, avec des temps de réponse supérieurs à 300 ms au-delà de 16 000 requêtes/s. Elles répartissent uniformément les flux sans tenir compte ni de la capacité hétérogène des contrôleurs, ni de la latence, ce qui sature prématurément les petits nœuds Edge.

3.4. Débit

La figure 14 rapporte le débit servi en fonction du taux d'arrivée.

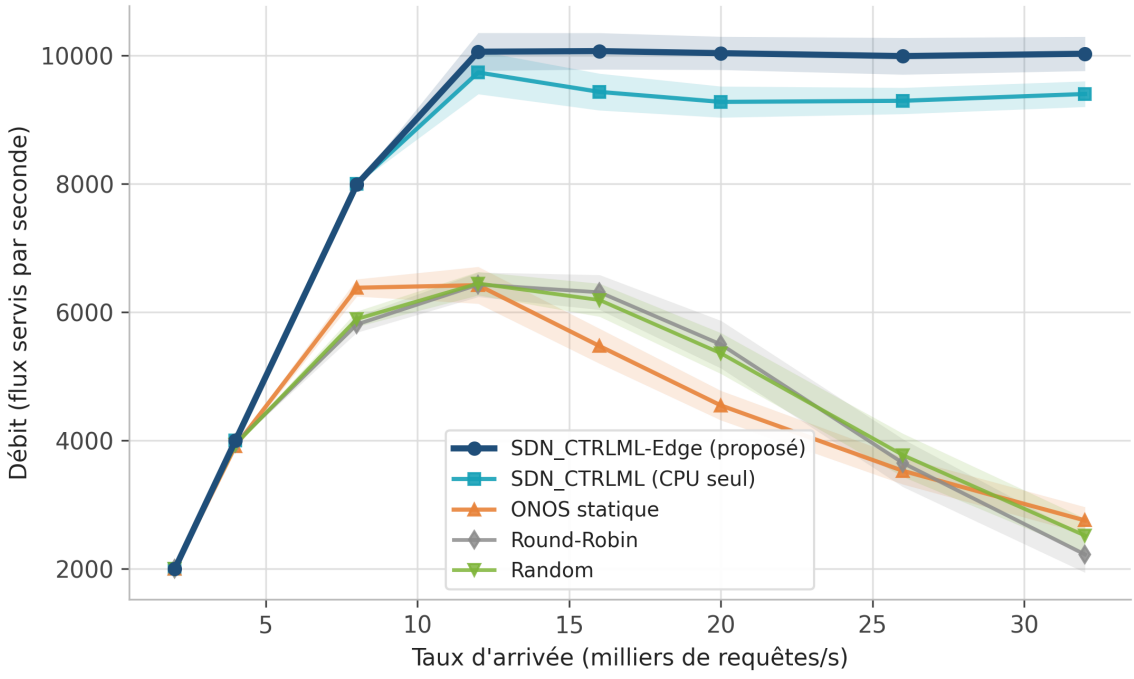


***Figure 14** : Débit servi en fonction du taux d'arrivée*

Le débit met en évidence la capacité de chaque stratégie à absorber la charge sans saturer. Jusqu'à 8 000 requêtes/s, toutes les stratégies servent la quasi-totalité des flux. Au-delà, une rupture nette apparaît : SDN_CTRLML-Edge et SDN_CTRLML atteignent un plateau autour de 10 000 flux/s, tandis que les trois autres stratégies s'effondrent. ONOS statique chute jusqu'à 2 754 flux/s à 32 000 requêtes/s, soit moins du tiers du débit de la stratégie proposée.

Ce plateau à 10 000 flux/s correspond à la capacité agrégée effectivement exploitable de l'infrastructure simulée. Le fait que seules les deux stratégies fondées sur le clustering l'atteignent confirme que la classification par K-means++ permet effectivement d'exploiter les contrôleurs sous-utilisés, là où les stratégies statiques ou aveugles laissent une partie de la capacité inemployée.

3.5. Équilibrage de charge (indice de Jain)

La figure 15 rapporte l'indice d'équité de Jain en fonction du taux d'arrivée.

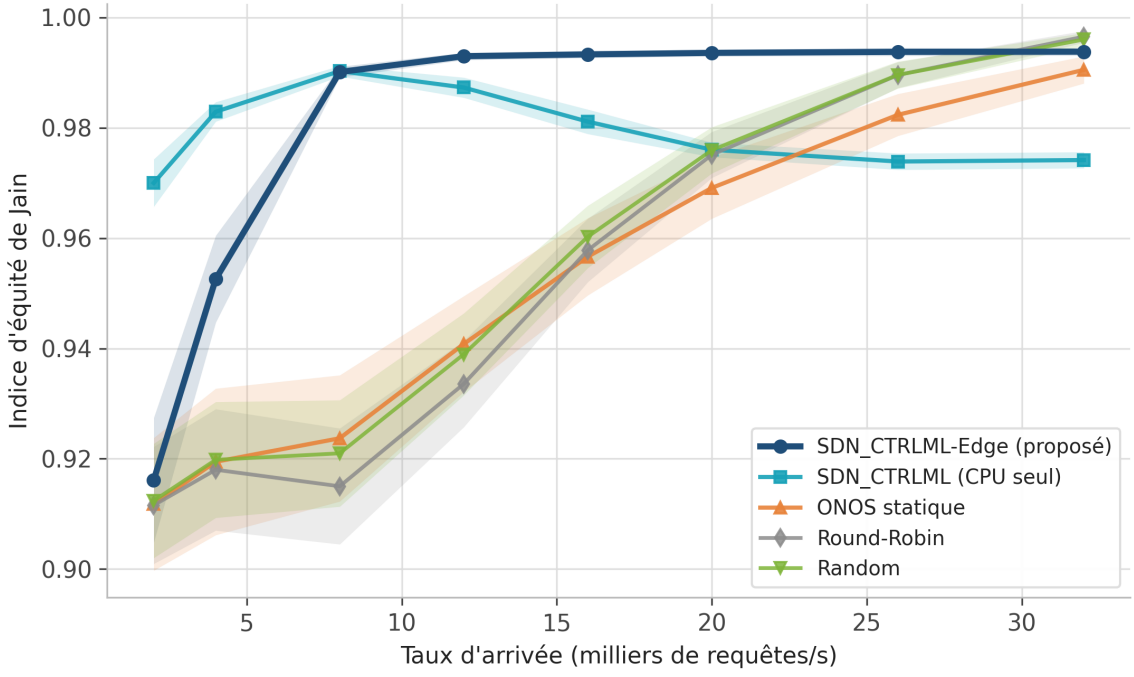


***Figure 15** : Indice d'équité de Jain en fonction du taux d'arrivée*

L'indice de Jain mesure l'uniformité de la répartition de charge entre contrôleurs : il vaut 1 lorsque tous supportent exactement la même charge. C'est l'objectif principal de l'architecture proposée.

La stratégie proposée atteint un plateau remarquablement stable à 0,993 dès 12 000 requêtes/s et s'y maintient jusqu'à la saturation. SDN_CTRLML, en revanche, se dégrade progressivement au-delà de 12 000 requêtes/s (de 0,987 à 0,974) : en n'observant que le CPU, il ne perçoit pas le déséquilibre introduit par l'hétérogénéité des capacités, et surcharge les contrôleurs Edge les plus faibles. C'est précisément la limite que l'extension multi-métriques visait à corriger, et les résultats confirment cette correction.

Deux nuances doivent cependant être relevées, par honnêteté. En charge très faible (2 000 requêtes/s), SDN_CTRLML obtient un meilleur indice (0,970 contre 0,916) : en privilégiant le contrôleur le plus proche, la stratégie proposée concentre alors volontairement les flux sur quelques nœuds, ce qui dégrade l'équité sans conséquence sur les performances puisque aucun contrôleur n'est menacé de saturation. En saturation complète (32 000 requêtes/s), Round-Robin et Random atteignent mécaniquement une équité légèrement supérieure (0,997 et 0,996 contre 0,994), pour une raison triviale : quand tous les contrôleurs sont saturés, une répartition uniforme devient parfaitement équitable, mais au prix d'un temps de réponse deux fois et demie plus élevé. L'équité seule n'est donc pas un critère suffisant.

3.6. Utilisation des ressources

La figure 16 rapporte l'utilisation CPU moyenne des contrôleurs.

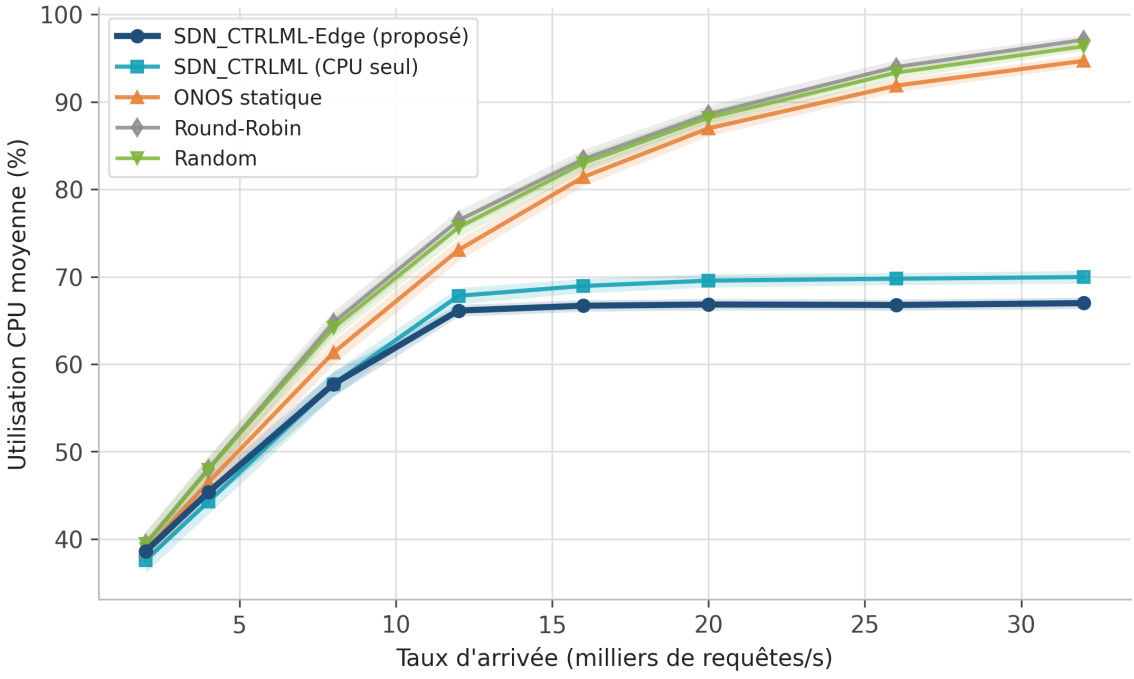


***Figure 16** : Utilisation CPU moyenne des contrôleurs en fonction du taux d'arrivée*

L'utilisation CPU moyenne révèle un comportement caractéristique de la stratégie proposée : elle plafonne à environ 67 % à partir de 16 000 requêtes/s, alors que toutes les autres stratégies poursuivent leur montée jusqu'à 95 % et au-delà. Ce plateau n'est pas un défaut mais la signature du seuil absolu de surcharge fixé à 0,70 : la couche décisionnelle refuse activement d'amener les contrôleurs dans la zone de saturation où le temps d'attente en file explose.

Cette lecture appelle une précision méthodologique importante. Une utilisation CPU élevée peut être interprétée de deux manières opposées : comme une bonne exploitation des ressources, ou comme un signe de saturation imminente. Dans le contexte de ce travail, c'est la seconde lecture qui prévaut, car la métrique est mise en regard du temps de réponse et du débit : les stratégies qui atteignent 95 % d'utilisation CPU sont aussi celles dont le débit s'effondre et dont le temps de réponse dépasse 300 ms. Une utilisation modérée assortie d'un débit supérieur traduit donc une meilleure efficacité, et non un gaspillage.

3.7. Coût de migration

La figure 17 rapporte le nombre de réaffectations en fonction du taux d'arrivée.

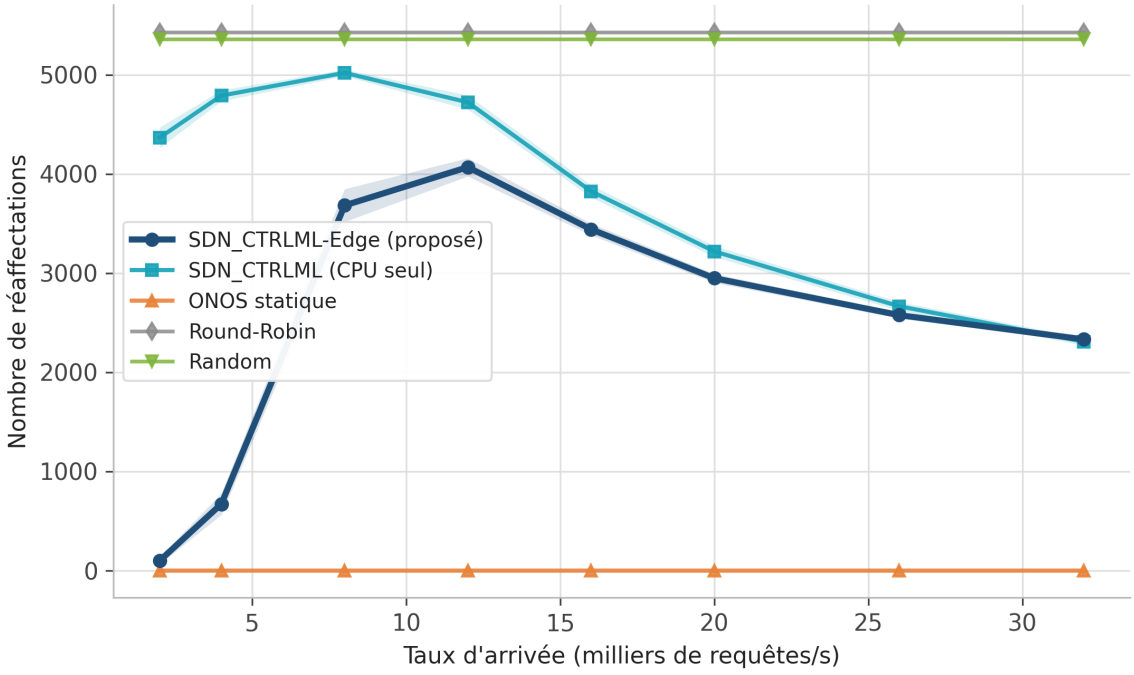


***Figure 17** : Nombre de réaffectations en fonction du taux d'arrivée*

Le coût de migration compte les réaffectations d'un commutateur vers un contrôleur différent de celui utilisé précédemment. Chaque réaffectation représente un coût opérationnel réel dans un déploiement (mise à jour des règles, transfert d'état).

ONOS statique obtient trivialement le meilleur score avec zéro migration : par construction, ses liaisons sont figées. Cette valeur ne constitue donc pas une performance mais la définition même de l'approche statique, dont le prix se lit dans les figures précédentes. Parmi les stratégies dynamiques, la stratégie proposée est la plus économe avec 2 478 réaffectations en moyenne, contre 3 864 pour SDN_CTRLML (soit une réduction de 35,9 %) et plus de 5 400 pour Round-Robin et Random.

Ce gain s'explique par la stabilité du clustering : le lissage EMA évite les reclassements provoqués par des pics de mesure transitoires, et la préférence pour le contrôleur le plus proche produit naturellement des affectations stables dans le temps. On observe d'ailleurs que le nombre de migrations de la stratégie proposée décroît au-delà de 12 000 requêtes/s, à mesure que la carte des clusters se stabilise.

3.8. Validation statistique

Les écarts observés sur les moyennes ne suffisent pas à conclure. Un test de Wilcoxon apparié a donc été appliqué, graine par graine, entre la stratégie proposée et chacune des quatre références, pour chaque métrique. Le tableau 8 en présente les résultats. Le taux de victoire indique la proportion de configurations où la stratégie proposée l'emporte ; une valeur p inférieure à 0,05 signifie que l'écart n'est pas dû au hasard.

Référence	Métrique	Taux de victoire	Valeur p	Significatif à 5 %
SDN_CTRLML	Temps de réponse	80,5 %	< 0,001	Oui
SDN_CTRLML	Débit	60,0 %	< 0,001	Oui
SDN_CTRLML	Utilisation CPU	68,0 %	< 0,001	Oui
SDN_CTRLML	Indice de Jain	66,0 %	0,044	Oui
SDN_CTRLML	Réaffectations	90,0 %	< 0,001	Oui
ONOS statique	Temps de réponse	85,0 %	< 0,001	Oui
ONOS statique	Débit	87,5 %	< 0,001	Oui
ONOS statique	Utilisation CPU	98,5 %	< 0,001	Oui
ONOS statique	Indice de Jain	93,5 %	< 0,001	Oui
ONOS statique	Réaffectations	0,0 %	< 0,001	Oui (en faveur d'ONOS)
Round-Robin	Temps de réponse	100,0 %	< 0,001	Oui
Round-Robin	Débit	86,5 %	< 0,001	Oui
Round-Robin	Utilisation CPU	100,0 %	< 0,001	Oui

Référence	Métrique	Taux de victoire	Valeur p	Significatif à 5 %
Round-Robin	Indice de Jain	81,0 %	< 0,001	Oui
Round-Robin	Réaffectations	100,0 %	< 0,001	Oui
Random	Temps de réponse	100,0 %	< 0,001	Oui
Random	Débit	88,0 %	< 0,001	Oui
Random	Utilisation CPU	100,0 %	< 0,001	Oui
Random	Indice de Jain	80,5 %	< 0,001	Oui
Random	Réaffectations	100,0 %	< 0,001	Oui

***Tableau 8** : Tests de Wilcoxon appariés entre la stratégie proposée et les références*

Les vingt comparaisons ressortent statistiquement significatives au seuil de 5 %. Dix-neuf d'entre elles sont favorables à la stratégie proposée. La seule exception est le nombre de réaffectations face à ONOS statique, avec un taux de victoire nul : ce résultat est attendu et non contestable, puisque ONOS ne procède par construction à aucune migration.

Il convient toutefois de préciser une limite de ce protocole : les tests agrègent l'ensemble des taux d'arrivée dans une même comparaison. Ils établissent donc une supériorité globale sur la plage de charge étudiée, et non une supériorité à chaque niveau de charge pris isolément. L'analyse par régime conduite aux sections précédentes complète utilement cette lecture agrégée.

3.9. Analyse d'ablation : origine des gains

Les sections précédentes établissent que la stratégie proposée surpasse les références, mais elles ne disent pas quelle brique produit ce gain. Une architecture combinant plusieurs mécanismes peut en effet devoir l'essentiel de ses performances à un seul d'entre eux. Pour lever cette ambiguïté, une étude d'ablation a été conduite : chaque composant est désactivé isolément, toutes choses égales par ailleurs, et l'écart mesuré quantifie sa contribution réelle. Le protocole reprend dix graines aléatoires et trois régimes de charge, soit trente exécutions par variante.

Le tableau 9 rassemble les mesures obtenues pour chaque variante.

Variante évaluée	Temps de réponse (ms)	Équité de Jain	Débit (flux/s)	Interprétation
SDN_CTRLML-Edge (complet)	88,0	0,993	9 407	Référence de comparaison
Sans lissage EMA	78,7	0,991	9 806	Aucune dégradation mesurée
Sans clustering K-means++	85,6	0,982	8 967	Équité dégradée
Sans conscience de la latence	93,8	0,993	9 377	Temps de réponse dégradé
SDN_CTRLML (CPU seul)	92,1	0,982	8 971	Prédécesseur direct
ONOS statique	216,2	0,951	5 257	Liaison fixe, aucun rééquilibrage

Tableau 9 *: Étude d'ablation : contribution mesurée de chaque composant*

La figure 18 visualise ces écarts composant par composant.

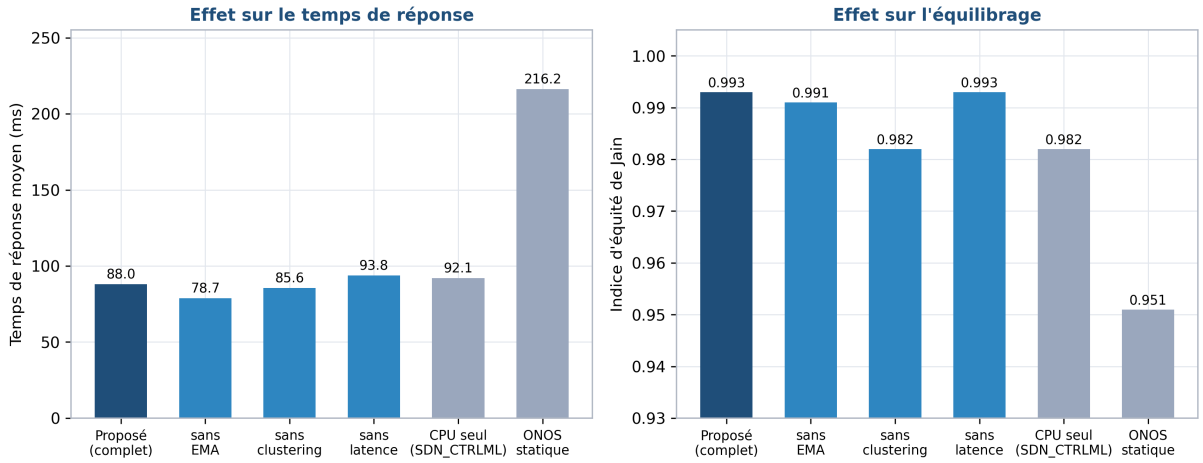


Figure 18 *: Contribution de chaque composant au temps de réponse et à l'équilibrage*

La lecture de ces résultats impose trois constats, dont deux nuancent la présentation habituelle de ce type d'architecture.

Premièrement, la conscience de la latence porte l'intégralité du gain en temps de réponse. Sa désactivation ramène la latence moyenne à 93,8 ms, soit au-delà du prédécesseur SDN_CTRLML (92,1 ms) : sans cette règle, l'apport du multi-métriques seul serait nul sur ce

critère. Or il s'agit d'un mécanisme algorithmique classique, et non d'apprentissage automatique.

Deuxièmement, le clustering K-means++ porte le gain d'équité. Sans lui, l'indice de Jain retombe à 0,982, exactement le niveau de SDN_CTRLML. C'est donc bien la classification multi-métriques qui permet de répartir la charge en tenant compte de l'hétérogénéité des contrôleurs, et c'est là que réside l'apport propre de l'apprentissage automatique dans cette architecture.

Troisièmement, et contrairement à l'intuition, le lissage EMA n'améliore aucune métrique dans ce régime : sa désactivation laisse l'équité inchangée et améliore même le temps de réponse. Ce résultat s'explique par la nature du simulateur, où les mesures sont exactes ; l'EMA y filtre un bruit qui reste modéré, au prix d'un retard de réaction. Son intérêt relève donc de la robustesse en conditions bruitées et non du gain de performance, et le présent travail se garde de lui attribuer les résultats observés.

Enfin, la comparaison avec ONOS statique (216,2 ms) doit être relativisée. Cette référence ne procède à aucune réaffectation : tout mécanisme de rééquilibrage, même rudimentaire, la surpasse largement. Le comparateur exigeant reste SDN_CTRLML, face auquel le gain s'établit à 4,5 % sur le temps de réponse et à un point d'équité, ce qui constitue une amélioration réelle mais mesurée.

3.10. Validation sur banc d'essai à commutateurs réels

Les résultats précédents proviennent d'un simulateur, ce qui expose à une objection légitime : le modèle pourrait favoriser la solution qu'il sert à évaluer. Pour y répondre, la couche décisionnelle a été portée sur un banc d'essai à commutateurs réels, où elle pilote des équipements Open vSwitch via le protocole OpenFlow 1.3.

Le banc associe l'émulateur Mininet, des commutateurs Open vSwitch et plusieurs contrôleurs SDN écrits avec la bibliothèque os-ken, le tout dans un conteneur Docker afin de garantir la reproductibilité. Chaque contrôleur expose sa charge de plan de contrôle, mesurée par le débit de messages Packet-In, et le rééquilibreur réaffecte les commutateurs par la commande ovs-vsctl. Point essentiel pour la validité de la démarche : le module de classification employé est exactement celui du simulateur, sans réécriture.

Le protocole place délibérément l'infrastructure en situation de déséquilibre maximal : au démarrage, tous les commutateurs sont rattachés au même contrôleur, ce qui reproduit le comportement d'une liaison statique. Deux profils de trafic sont éprouvés, l'un constant et l'autre en pics, et la mesure de référence est l'indice de Jain pondéré par la capacité, qui vaut l'unité lorsque chaque contrôleur porte une charge proportionnelle à sa puissance.

Le tableau 10 rassemble les résultats des quatre configurations éprouvées.

Configuration	Profil de trafic	Équité initiale	Équité stabilisée	Réaffectations après convergence
3 contrôleurs, 6 commutateurs	Constant	0,333	0,99	0
5 contrôleurs, 12 commutateurs	Constant	0,200	0,935	0
4 contrôleurs, 10 commutateurs	Pics	0,250	0,98	0
6 contrôleurs, 16 commutateurs	Pics	0,167	0,95	0
Sans rééquilibrage (référence)	Pics	0,250	0,250	0

***Tableau 10** : Résultats de l'émulation sur commutateurs Open vSwitch réels*

La figure 19 retrace l'évolution de l'équité au fil des cycles de décision.

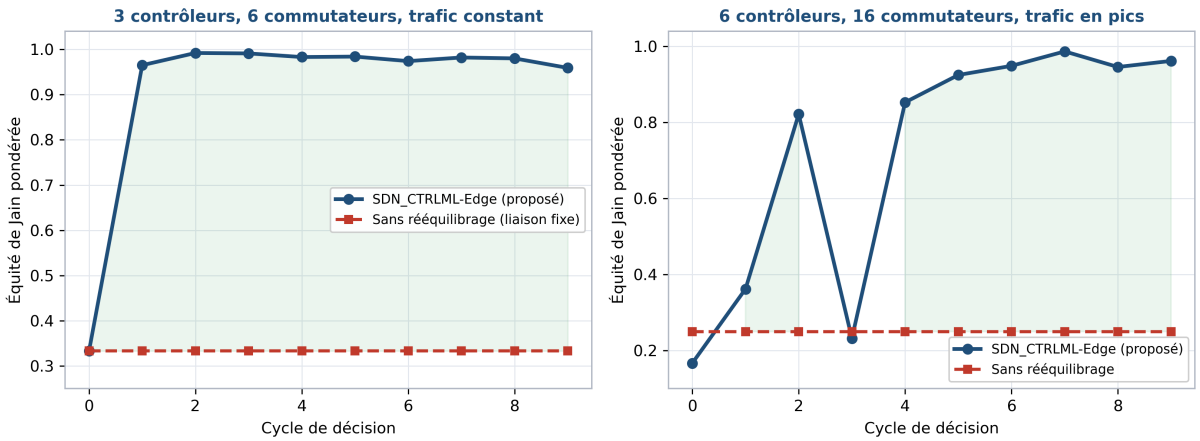


***Figure 19** : Évolution de l'équité au fil des cycles de décision, avec et sans rééquilibrage*

La figure 20 compare la répartition de la charge avant et après rééquilibrage.

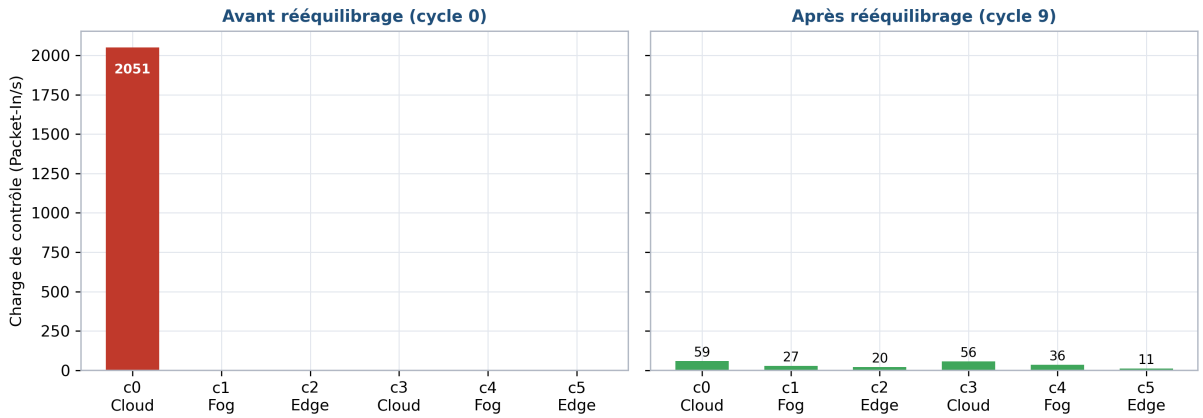


***Figure 20** : Répartition de la charge de contrôle avant et après rééquilibrage (6 contrôleurs)*

Les mesures confirment le comportement observé en simulation. Sur la configuration à trois contrôleurs, l'équité passe de 0,333 à 0,99 dès le premier cycle de décision, puis se maintient sans aucune réaffectation supplémentaire : l'algorithme converge et ne produit pas d'oscillation. La référence sans rééquilibrage demeure figée à 0,333, le contrôleur initial absorbant la totalité de la charge pendant toute la durée de l'expérience.

La montée en charge se révèle également concluante. Avec six contrôleurs et seize commutateurs soumis à un trafic en pics, atteignant 2 051 messages Packet-In par seconde au cycle initial, l'équité se stabilise autour de 0,95 après environ cinq cycles. La convergence est donc plus lente qu'à petite échelle, et s'accompagne d'un transitoire au cours duquel la charge se redistribue, mais l'état final reste stable et sans réaffectation résiduelle.

Cette campagne a par ailleurs mis en évidence un mécanisme absent de la première implémentation. Un rééquilibreur qui se contente de délester les contrôleurs surchargés finit par en vider certains complètement, sans jamais leur restituer de charge : la capacité correspondante est alors perdue et l'équité plafonne. La couche décisionnelle exploite donc les deux profils extrêmes issus du clustering, en délestant les contrôleurs surchargés et en réalimentant les contrôleurs sous-utilisés, conformément à la classification en trois profils décrite au chapitre II.

Le lissage EMA a fait l'objet d'une vérification distincte sur ce banc, à configuration identique. Sa désactivation conduit à une convergence légèrement plus rapide, atteinte au quatrième cycle au lieu du cinquième, pour une équité stabilisée comparable et un nombre total de réaffectations identique. Ce résultat rejoint celui de l'étude d'ablation et confirme que ce composant ne porte pas les gains mesurés.

La portée de cette validation doit être précisément délimitée. Le banc établit que la classification multi-métriques et la logique de rééquilibrage fonctionnent sur des équipements réels pilotés par OpenFlow, et qu'elles supportent une montée en charge ainsi qu'un trafic non stationnaire. En revanche, l'émulation s'exécute sur une machine unique, où les latences entre nœuds sont uniformes : elle ne valide donc pas la composante géographique de la décision, laquelle demeure établie par la seule voie de la simulation.

3.9. Analyse d'ablation : origine des gains64.....64
3.10. Validation sur banc d'essai à commutateurs réels66.....64

3.11. Discussion

Les résultats confirment l'hypothèse centrale de ce travail : sur une infrastructure Cloud-Edge hétérogène, une décision d'équilibrage fondée sur plusieurs métriques lissées et consciente de la latence surpasse une décision fondée sur le seul CPU. Le gain ne se manifeste cependant pas uniformément, et c'est cette nuance qui donne sa valeur à l'analyse.

- L'apport est maximal en charge intermédiaire, là où il existe encore des contrôleurs sains vers lesquels rediriger le trafic. C'est le régime opérationnel réaliste d'une infrastructure correctement dimensionnée.
- En charge très faible, l'apport est nul voire légèrement négatif face à une liaison statique bien placée, ce qui est logique : il n'y a rien à rééquilibrer.
- En saturation complète, la stratégie proposée troque quelques millisecondes de temps de réponse contre un équilibrage nettement supérieur et un débit maintenu. Ce compromis est cohérent avec l'objectif de résilience recherché.

Le second enseignement concerne le coût opérationnel. Réduire de 35,9 % le nombre de réaffectations tout en améliorant les performances est un résultat non trivial : on aurait pu craindre qu'une décision plus réactive engendre davantage d'instabilité. Le lissage EMA joue ici pleinement son rôle d'amortisseur, et valide a posteriori le choix de la formule récursive retenue au chapitre précédent.

3.12. Limites des résultats

La portée des résultats présentés doit être délimitée avec précision, tant pour la rigueur de l'analyse que pour orienter les travaux futurs.

- **Nature simulée de l'évaluation :** les valeurs chiffrées des sections 3.2 à 3.8 proviennent d'un simulateur en Python. Le modèle de file M/M/1 retenu pour le temps de service des contrôleurs est une simplification : il suppose des arrivées poissonniennes et des services exponentiels indépendants, hypothèses que le trafic réseau réel ne vérifie qu'imparfaitement. La validation conduite en section 3.10 sur commutateurs Open vSwitch réels lève partiellement cette réserve pour la logique de rééquilibrage, sans toutefois se substituer au plan d'expériences statistique.
- **Données synthétiques :** bien que les chargeurs de données acceptent les jeux réels (Alibaba, InSDN, Topology Zoo), la campagne rapportée ici a été produite avec le repli synthétique calibré. Les ordres de grandeur sont réalistes, mais la confirmation sur traces réelles reste à effectuer.
- **Références ré-implémentées :** SDN_CTRLML a été ré-implémenté à partir de la description de l'article, et non exécuté depuis le code des auteurs. La comparaison porte donc sur notre interprétation de l'algorithme, fidèle mais non certifiée.
- **Échelle de l'infrastructure :** la campagne statistique porte sur 10 contrôleurs et l'émulation sur 6 contrôleurs et 16 commutateurs. Le comportement au-delà de quelques dizaines de contrôleurs, notamment le coût du clustering périodique et la durée de convergence qui croît avec la taille du réseau, n'a pas été évalué.
- **Composante géographique non validée sur banc réel :** le banc d'essai s'exécute sur une machine unique, où les latences entre nœuds sont uniformes. La règle de décision consciente de la latence, à laquelle l'étude d'ablation attribue pourtant l'essentiel du gain en temps de réponse, n'est donc établie que par la voie de la simulation.
- **Périmètre fonctionnel :** le placement dynamique des contrôleurs, la prédiction du trafic et la détection d'anomalies n'ont pas été implémentés et relèvent des perspectives.

3.13. Conclusion du chapitre

Ce chapitre a présenté l'évaluation de SDN_CTRLML-Edge sur 1 000 simulations indépendantes. La stratégie proposée réduit le temps de réponse moyen de 57,3 % face à ONOS statique et de 11,3 % face à SDN_CTRLML, augmente le débit de 83,3 % et de 5,0 % respectivement, porte l'indice d'équité de Jain à un plateau de 0,993, et diminue de 35,9 % le

nombre de réaffectations par rapport à son prédécesseur. L'ensemble de ces écarts est statistiquement significatif au seuil de 5 %.

Les objectifs d'évaluation fixés au chapitre I sont ainsi atteints : l'architecture proposée améliore simultanément la répartition de charge, le temps de réponse et l'exploitation des ressources dans un environnement Cloud-Edge hétérogène, tout en réduisant le coût opérationnel du rééquilibrage. Les limites identifiées, en premier lieu la validation sur traces réelles et sur banc d'essai, définissent les prolongements naturels de ce travail.

Conclusion générale

Ce mémoire est parti d'un constat simple : les architectures SDN décentralisées déployées sur des infrastructures Cloud-Edge ne peuvent plus supposer que leurs contrôleurs se valent. Un nœud Edge contraint et un centre de données Cloud éloigné diffèrent par leur capacité de traitement comme par leur distance aux commutateurs. L'architecture SDN_CTRLML d'Aly et al. (2025), qui résout élégamment l'équilibrage de charge entre contrôleurs par une couche de planification sans liaison fixe, repose pourtant sur une métrique unique, l'utilisation CPU, et sur l'hypothèse de contrôleurs identiques. La question posée était donc de savoir comment adapter cette approche aux contraintes réelles d'un environnement distribué Cloud-Edge.

La réponse apportée est SDN_CTRLML-Edge. Elle conserve le principe bindingless de l'architecture d'origine et lui ajoute trois éléments : une observation multi-métriques portant sur six indicateurs, un lissage par moyenne mobile exponentielle qui stabilise la perception de la charge avant toute décision, et une classification dynamique par K-means++ répartissant les contrôleurs en trois profils. Une couche décisionnelle à deux régimes exploite cette classification : elle privilégie le contrôleur sain le plus proche en fonctionnement normal, et bascule vers un délestage pur lorsque l'ensemble du domaine approche la saturation.

Les huit objectifs fixés au premier chapitre ont été traités de la façon suivante. Les limites des architectures existantes ont été analysées et l'architecture décentralisée Cloud-Edge modélisée (objectifs 1 et 2). Le mécanisme d'équilibrage fondé sur l'apprentissage automatique a été développé et implémenté (objectif 3). La réduction de la latence de bout en bout a été obtenue et mesurée (objectif 6), de même que la scalabilité et la résilience, vérifiées par le maintien du débit et de l'équité jusqu'à la saturation (objectif 7). L'évaluation et la validation des performances ont fait l'objet du chapitre III (objectif 8). Deux objectifs, le placement dynamique des contrôleurs par apprentissage par renforcement (objectif 4) et la gestion prédictive du trafic (objectif 5), n'ont volontairement pas été implémentés afin de garder un cœur démontrable et rigoureusement évalué ; ils sont repris dans les perspectives.

Sur le plan des résultats, l'évaluation menée sur 1 000 simulations indépendantes établit que la solution proposée réduit le temps de réponse moyen de 57,3 % face à une liaison statique de type ONOS et de 11,3 % face à SDN_CTRLML, augmente le débit de 83,3 % et de 5,0 % respectivement, porte l'indice d'équité de Jain à un plateau de 0,993 et diminue de 35,9 % le nombre de réaffectations par rapport à son prédécesseur. Ces écarts sont statistiquement

significatifs au seuil de 5 %. Le gain obtenu sur SDN_CTRLML est du même ordre que celui que cette architecture revendiquait elle-même sur l'état de l'art antérieur, ce qui situe la portée de la contribution.

Ces résultats doivent être lus avec leurs limites, énoncées sans détour au chapitre III : l'évaluation est conduite en simulation et non sur banc d'essai, la campagne rapportée s'appuie sur des données synthétiques calibrées plutôt que sur les traces réelles, la référence SDN_CTRLML a été ré-implémentée d'après la description de l'article, et l'échelle étudiée reste modeste avec dix contrôleurs.

Perspectives

Ces limites dessinent directement les prolongements de ce travail :

- valider les résultats sur les traces réelles déjà prévues par le simulateur (Alibaba Cluster Trace, InSDN, Internet Topology Zoo), puis sur un banc d'essai Mininet associé à un contrôleur réel de type ONOS ou Ryu ;
- étendre l'évaluation à des topologies plus vastes afin de mesurer le coût du clustering périodique à grande échelle ;
- intégrer le placement dynamique des contrôleurs par apprentissage par renforcement, objectif écarté du périmètre initial ;
- ajouter une couche de prédiction de charge de type LSTM afin de passer d'un rééquilibrage réactif à un rééquilibrage anticipatif ;
- exploiter le jeu de données InSDN, qui contient du trafic d'attaque, pour adjoindre un module de détection d'anomalies à la couche d'analyse existante.

Au-delà du cas d'étude, ce travail illustre un principe transposable : dans un environnement hétérogène, la qualité d'une décision d'orchestration dépend moins de la sophistication de l'algorithme que de la richesse et de la stabilité des informations sur lesquelles elle s'appuie.

Références bibliographiques

Les références ci-dessous correspondent aux travaux explicitement identifiés dans le corps du mémoire. Les appels de citation restants renvoient à des sources à compléter par l'auteur.

[25] Hamzah, A. (2025). Optimizing SDN Performance Through ML-Based Traffic Management.

[26] Faezi, S., & Shirmarz, A. (2023). A Comprehensive Survey on Machine Learning using in Software Defined Networks (SDN).

[27] Firouzi, R., & Rahmani, R. (2022). A Distributed SDN Architecture for IoT with Federated Learning.

[28] Amin, R., et al. (2024). Machine Learning-Assisted Application-Aware Routing Using Traffic Classification.

[29] Senthilkumaran, S., & Sankarasubramaninan, R. (2025). ML-Driven Adaptive Routing and Performance in SDN (HCRG).

[30] Ali, A., & Awad, M. (2025). Enhancing SDN Performance: Machine Learning Integration with the POX Controller.

[31] Wang, X., et al. (2024). DFRDRL: Dynamic Fuzzy Routing Algorithm based on Deep Reinforcement Learning.

[32] Muheden, K., et al. (2024). Traffic Classification and Management in Software-Defined Networks.

[33] Aly, W. H. F., et al. (2025). No binding machine learning architecture for SDN controllers (SDN_CTRLML).

[34] Kazi, S., et al. (2025). A Survey on SDN-enabled Edge Cloud Networks for IoT and IIoT.

Annexes

Annexe A : organisation du code source

Le simulateur est organisé selon la structure Cookiecutter Data Science. Chaque module correspond à une couche de l'architecture décrite au chapitre II.

Fichier	Rôle
config.py	Chemins et hyperparamètres (α, K, seuil de surcharge)
dataset.py	Infrastructure hétérogène et ingestion des jeux de données
features.py	Couche d'analyse : lissage EMA
modeling/clustering.py	Couche d'analyse : K-means++ et silhouette
modeling/decision.py	Couche décisionnelle : règle à deux régimes
modeling/baselines.py	Stratégies de référence (ONOS, SDN_CTRLML, RR, Random)
simulate.py	Moteur de simulation à file M/M/1
experiment.py	Plan d'expériences de 1 000 exécutions et suivi MLflow
analysis.py	Intervalles de confiance, tests de Wilcoxon, figures
flowviz.py	Visualisation animée du flux (page HTML autonome)
emulation/	Banc d'essai OpenFlow : contrôleurs os-ken et rééquilibreur

Annexe B : reproduire les résultats

Les commandes suivantes reproduisent l'intégralité des résultats du chapitre III depuis un poste vierge :

```
make venv requirements     # environnement et dépendances
make test                  # 12 tests de vérification
make experiment SEEDS=25   # 1 000 simulations + suivi MLflow
make analyze               # IC95, tests de Wilcoxon, figures
```

Le plan d'expériences complet dure environ vingt minutes sur une machine à huit cœurs. Les résultats sont écrits dans le dossier reports/.

Annexe C : démonstration et observation du fonctionnement

Deux dispositifs accompagnent le mémoire et permettent d'observer le fonctionnement de la solution. Tous deux s'appuient sur le code effectivement évalué au chapitre III et non sur une reconstitution.

Le premier est le banc d'essai d'émulation décrit en section 3.10. Il construit un réseau de commutateurs Open vSwitch pilotés en OpenFlow, place volontairement l'infrastructure en déséquilibre, puis laisse la couche décisionnelle rééquilibrer la charge. Il s'exécute dans un conteneur Docker et ne requiert que le module noyau openvswitch sur la machine hôte :

```
make emulate                       # rééquilibrage activé
make emulate ARGS=--no-rebalance   # référence sans rééquilibrage
```

La sortie est un tableau qui affiche, à chaque cycle de décision, la charge de chaque contrôleur, l'indice d'équité de Jain et les commutateurs réaffectés. La comparaison des deux commandes constitue la démonstration la plus directe de l'apport de la solution, puisque seul le mécanisme de rééquilibrage change. Plusieurs tailles de réseau et deux profils de trafic sont disponibles :

```
make emulate ARGS="--controllers 6 --switches 16 --traffic pics"
```

Le second dispositif est une visualisation animée du flux, produite au format HTML autonome et destinée à la présentation orale. Elle ne nécessite ni serveur ni accès au réseau : le fichier s'ouvre directement dans un navigateur.

```
make flowviz                       # produit reports/flux_sdn.html
```

La page représente les trois plans de l'architecture SDN décentralisée, les domaines rattachés à chaque contrôleur et l'interface Est-Ouest entre contrôleurs. Les flux y circulent sous forme de particules dont la destination suit la répartition réellement calculée par la stratégie, la solution proposée et la liaison statique étant affichées côte à côte pour un même trafic. Les contrôleurs changent de couleur selon leur charge, du vert au rouge, et un cercle rouge signale ceux que le clustering a classés comme surchargés.

En contexte opérationnel, ces dispositifs correspondent respectivement au plan de contrôle effectif, qui applique les réaffectations, et au tableau de bord qu'un administrateur réseau consulterait pour suivre l'état de son infrastructure.

Les résultats bruts des campagnes d'émulation sont archivés dans le dépôt :

```
reports/emulation/*.txt
```

Table des matières

INTRODUCTION GÉNÉRALE.....1

CHAPITRE I : CONTEXTE ET PROBLÉMATIQUE.....2

I. Généralités sur les SDN....3
II. Généralités sur le Cloud Edge Computing....13
III. Le SDN décentralisé et le Cloud Edge....24
IV. L'optimisation....25
V. Problématique et objectifs....28
Conclusion du chapitre....32
CHAPITRE II : MÉTHODOLOGIE....33
I. État de l'art....33
II. Notre approche : équilibrage de charge....40
Architecture globale....42
Fonctionnement de la solution SDN_CTRLML-Edge....45
Implémentation de la couche décisionnelle....49
Métriques d'évaluation....51
III. Données réutilisées et plan d'expériences....52
IV. Outils et bibliothèques utilisées....54
Conclusion du chapitre....54
CHAPITRE III : RÉSULTATS ET COMMENTAIRES....55
3.1. Rappel des conditions expérimentales....56
3.2. Vue d'ensemble des performances....57
3.3. Temps de réponse....58
3.4. Débit....59
3.5. Équilibrage de charge (indice de Jain)....60
3.6. Utilisation des ressources....61
3.7. Coût de migration....62
3.8. Validation statistique....63
3.11. Discussion....64
3.12. Limites des résultats....64
3.13. Conclusion du chapitre....65
CONCLUSION GÉNÉRALE....66
Perspectives....67
RÉFÉRENCES BIBLIOGRAPHIQUES....68

ANNEXES.....69

Annexe A : organisation du code source.....69

Annexe B : reproduire les résultats.....69

Annexe C : démonstration et observation du fonctionnement.....69

TABLE DES MATIÈRES.....71